

1	Templates and Scripts	1	7.7	Manacher	12
1.1	sublime	1	7.8	Aho Corasick	12
1.2	vs code	1	7.9	Min Lexic Rotation	12
1.3	Interactor	1	7.10	Longest Palindrome	13
1.4	debug.h	1	7.11	Wildcard Matching	13
2	Number Theory	1	7.12	Hashing	13
2.1	mini mod	1	7.13	2D Hash	13
2.2	Binomial Coefficient	1	8	Numerical	13
2.3	Sieve, SPF, Phi	1	8.1	FFT	13
2.4	Segmented Sieve	1	8.2	FFT With mod M	14
2.5	Safe LCM	2	8.3	NTT	14
2.6	Extended GCD	2	8.4	FWHT	14
2.7	CRT	2	9	Linear Algebra	14
2.8	Diophantine Equation	2	9.1	The Matrix Class	14
2.9	Pollar Rho Factorization	2	9.2	Gaussian Elimination	15
3	Dynamic Programming	3	9.3	Determinant	15
3.1	SOS DP	3	9.4	Matrix Inverse	15
3.2	CHT	3	9.5	Xor Basis	15
3.3	CHT Dynamic	3	10	Geometry	15
4	Data Structure	3	10.1	Full Template	15
4.1	Fenwick Tree	3	11	Miscellaneous	19
4.2	Fenwick 2D (Compressed)	3	11.1	Interval Container	19
4.3	BIT 2D	4	11.2	Efficient Mask Traversal	19
4.4	Segment Tree	4	11.3	Generating Permutations	19
4.5	Lazy Propagation	4	11.4	Iterating over Submasks	19
4.6	Persistent Segment Tree	4	11.5	Random	19
4.7	Merge Sort Tree	5	11.6	Index compression	19
4.8	Merse Sort Tree with Update	5	11.7	pbds	19
4.9	Treap	5	11.8	Bitset	19
4.10	Segment Tree 2D (1 based)	6	11.9	Primes	19
4.11	Segment Tree 2D (Index compression)	6	11.10	Digit Range Sum	19
4.12	DSU	6	11.11	Custom Hash	19
4.13	SQRT Decomposition	6	12	Formulae	19
4.14	MOs Algorithm	7	1	Templates and Scripts	
4.15	MOs with updates	7	1.1	sublime	
4.16	Treap	7	{		
4.17	Wavelet Tree	7	"shell_cmd": "g++ -Wall -std=c++20 -O2		
4.18	Sparse Table (RMQ)	8	-DLOCAL \$file -o \$file_base_name		
4.19	Prefix Sum 2D	8	&& bash -c 'ulimit -S -f 10000;		
4.20	Prefix Sum 3D	8	timeout 5 ./ \$file_base_name < input		
5	Graph	8	.in > output.out 2> error.log'",		
5.1	Dijkstra	8	"file_regex": "~([~:]*)([0-9]+)		
5.2	Bellmanford	8	:?([0-9]+)?:(.*)\$",		
5.3	BCC, Bridge, AP	8	"working_dir": "\$file_path",		
5.4	2CC	8	"selector": "source.cpp",		
5.5	LCA	9	// sanitize flags: -fsanitize=address		
5.6	SCC	9	-fsanitize=undefined		
5.7	Kosaraju	9	}		
5.8	HLD	9	1.2	vs code	
5.9	Centroid Decomposition	10	{		
5.10	Arpa's Trick	10	"version": "2.0.0",		
6	Flow and Matching	10	"tasks": [{		
6.1	Min Cost Max Flow	10	"label": "cp",		
6.2	Dinic	10	"type": "shell",		
6.3	HopcroftKarp	11	"command": "g++ -Wall -std=c++20 -O2		
7	String	11	-DLOCAL \"\${file}\" -o \"\${file		
7.1	Trie	11	Dirname}/\${file		
7.2	Bit Trie	11	BaseNameNoExtension}\" && \"\${file		
7.3	Z Algorithm	11	Dirname}/\${file		
7.4	KMP	11	BaseNameNoExtension}\" < input.		
7.5	Suffix Array	11	in > output.out 2> error.log",		
7.6	Suffix Automaton	12	"group": "build",		
			"presentation": { "reveal": "silent"		
			},		
			"problemMatcher": ["\$gcc"]		
			}]		

1.3 Interactor

```
socat EXEC:"./interactor" EXEC:"./soln"
```

1.4 debug.h

```
#include <iostream>
using namespace std;
#define see(x) cerr << #x << ": " << x
               << nl
template <typename... Args>
void debug(Args... args) {
    ((cerr << " " << args), ...) << "\n";
}

```

2 Number Theory

2.1 mini mod

```
inline ll mod(ll n, ll m = MOD) { return
    (m + n % m) % m; }
ll modpow(ll n, ll k, ll m = MOD) {
    ll ret = 1;
    n = mod(n, m);
    while (k) {
        if (k & 1) ret = (ret * n) % m;
        n = (n * n) % m;
        k >>= 1;
    }
    return ret;
}
inline ll inv(ll n, ll m = MOD) { return
    modpow(n, m - 2, m); }
inline ll mul(ll x, ll y, ll m = MOD) {
    return (mod(x, m) * mod(y, m)) % m;
}
inline ll dvd(ll x, ll y, ll m = MOD) {
    return mul(x, inv(y, m), m); }

```

2.2 Binomial Coefficient

```
ll fact[MAXN];
void calc_fact(ll m = MOD) {
    fact[0] = 1;
    for (int i = 1; i < MAXN; ++i) {
        fact[i] = (fact[i - 1] * i) % m;
    }
}
ll C(int n, int k, ll m = MOD) {
    if (k > n) return 0;
    return dvd(fact[n], fact[k] * fact[n -
        k] % m, m);
}

```

2.3 Sieve, SPF, Phi

```
vector<int> primes;
bool isprime[MAXN];
int spf[MAXN], phi[MAXN];
void sieve() {
    for (int i = 0; i < MAXN; ++i) {
        isprime[i] = true; spf[i] = i; phi
        [i] = i;
    }
    isprime[0] = isprime[1] = false;
    for (ll i = 2; i < MAXN; ++i) {
        if (isprime[i]) {
            primes.push_back(i); phi[i] =
            i - 1;
            for (ll j = 2 * i; j < MAXN; j
            += i) {
                isprime[j] = false;
                if (spf[j] == j) spf[j] =
                i;
                phi[j] -= phi[j] / i;
            }
        }
    }
}

```

2.4 Segmented Sieve

```
// Generate all primes from l to r using
    segmented sieve in
```

```
//  $O((r-l) \log(r) + \sqrt{r})$  // needs
// sieve(limit)
vector<int64_t> segmented_sieve(int64_t
    l, int64_t r) {
    if (l == 1) l++;
    int limit = sqrtl(r);
    while (limit * limit <= r) limit++;
    while (limit * limit > r) limit--;
    auto primes = sieve(limit);
    vector<bool> is_prime(r - l + 1, true);
    for (int64_t p : primes) {
        int64_t start = max(p * p, (l + p - 1) / p * p);
        for (int64_t j = start; j <= r; j += p) {
            is_prime[j - l] = false;
        }
    }
    vector<int64_t> vec;
    for (int64_t i = l; i <= r; ++i) {
        if (is_prime[i - l]) vec.push_back(i);
    }
    return vec;
}
```

2.5 Safe LCM

Description: Finds lcm of (a, b) where their lcm cannot go above lim . If it goes above lim , it simply returns $lim + 1$.

```
__int128 safe_lcm(__int128 a, __int128 b
    , __int128 lim) {
    __int128 t = ((a) / gcd(a, b));
    if (t > (lim / b)) return lim+1;
    else return t * b;
}
```

2.6 Extended GCD

Description: Finds solutions (x_0, y_0) in $ax+by = 1$. (Check formulae for more details)

```
int extended_gcd(ll a, ll b, ll& x, ll&
    y) {
    if (!b) {
        x = 1, y = 0;
        return a;
    }
    ll g = extended_gcd(b, a % b, x, y);
    swap(x, y);
    y -= x * (a / b);
    return g;
}
```

2.7 CRT

Description: Computes the unique solution x to a system of congruences $x \equiv a_i \pmod{m_i}$ using the Chinese Remainder Theorem. Assumes all moduli are pairwise coprime and the answer fits in ll .

Usage: Store congruences as $\{\text{remainder}, \text{modulus}\}$ pairs in mods . Call $\text{crt}(\text{mods})$ to obtain the smallest non-negative solution modulo the product of all moduli. Requires $\text{extended_gcd}()$ and $\text{modinv}()$.

Time: $O(k \log M)$

```
ll modinv(ll a, ll m) {
    ll x, y;
    extended_gcd(a, m, x, y);
    x %= m;
    return x < 0 ? x + m : x;
}
ll crt(vector<pair<ll, ll>>& mods) {
    ll mod = 1;
    for (auto& p : mods) mod *= p.ss;
    ll ans = 0;
    for (auto [x, p] : mods) {
        ll m = mod / p;
        m = (m * modinv(m, p)) % mod;
        m = (m * x) % mod;
        ans = (ans + m) % mod;
    }
    return ans;
}
```

2.8 Diophantine Equation

Description: Solves $ax+by = c$ where $\gcd(a, b) | c$. Find any solution returns $x \in [\text{minx}, \text{maxx}]$ and $y \in [\text{miny}, \text{maxy}]$. If $\text{solves} = \text{true}$, the function returns all solution pairs (x, y) . (Check formulae for more details)

```
// Solves  $a*x + b*y = c$ 
bool find_any_solution(ll a, ll b, ll c, ll
    &x0, ll &y0, ll &g) {
    if (a==0&&b==0) {
        if (c) return false;
        x0=y0=g=0; return true;
    }
    g=extended_gcd(abs(a), abs(b), x0, y0);
    if (c%g!=0) return false;
    x0*=c/g; y0*=c/g;
    if (a<0) x0*=-1;
    if (b<0) y0*=-1;
    return true;
}
// Shift solution by cnt
void shift_solution(ll &x, ll &y, ll a, ll
    b, ll cnt) {
    x+=cnt*b; y-=cnt*a;
}
pair<ll, vector<pair<ll, ll>>>
    find_all_solutions(ll a, ll b, ll c,
        ll minx, ll maxx, ll
        miny, ll maxy, bool solves) {
    ll x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return {0, {}};
    if (a==0&&b==0) {
        assert(c==0);
        ll cnt = 1LL*(maxx-minx+1)*(maxy-
            miny+1);
        vector<pair<ll, ll>> res;
        if (solves) {
            for (ll cur_x=minx; cur_x<=
                maxx; ++cur_x) {
                for (ll cur_y=miny; cur_y
                    <=maxy; ++cur_y) {
                    res.push_back({cur_x
                        , cur_y});
                }
            }
            return {cnt, res};
        }
        if (a==0) {
            ll cnt = (maxx-minx+1)*(miny<=c/
                b&&c/b<=maxy);
            vector<pair<ll, ll>> res;
            if (solves && cnt > 0) {
                ll cur_y = c/b;
                for (ll cur_x=minx; cur_x<=
                    maxx; ++cur_x) {
                    res.push_back({cur_x,
                        cur_y});
                }
            }
            return {cnt, res};
        }
        if (b==0) {
            ll cnt = (maxy-miny+1)*(minx<=c/
                a&&c/a<=maxx);
            vector<pair<ll, ll>> res;
            if (solves && cnt > 0) {
                ll cur_x = c/a;
                for (ll cur_y=miny; cur_y<=
                    maxy; ++cur_y) {
                    res.push_back({cur_x,
                        cur_y});
                }
            }
            return {cnt, res};
        }
        ll orig_a = a, orig_b = b;
        a/=g; b/=g;
        ll sign_a=a>0?1:-1, sign_b=b>0?1:-1;
        shift_solution(x, y, a, b, (minx-x)/b);
        if (x<minx) shift_solution(x, y, a, b,
            sign_b);
        if (x>maxx) return {0, {}};
        ll lx1=x;
        shift_solution(x, y, a, b, (maxx-x)/b);
        if (x>maxx) shift_solution(x, y, a, b, -
            sign_b);
        ll rx1=x;
        shift_solution(x, y, a, b, -(miny-y)/a);
        if (y<miny) shift_solution(x, y, a, b, -
            sign_a);
        if (y>maxy) return {0, {}};
        ll lx2=x;
        shift_solution(x, y, a, b, -(maxy-y)/a);
        if (y>maxy) shift_solution(x, y, a, b,
            sign_a);
        ll rx2=x;
        if (lx2>rx2) swap(lx2, rx2);
        ll lx=max(lx1, lx2), rx=min(rx1, rx2);
        if (lx>rx) return {0, {}};
        ll cnt = (rx-lx)/abs(b)+1;
        vector<pair<ll, ll>> res;
        if (solves) {
            for (ll cur_x=lx; cur_x<=rx;
                cur_x+=abs(b)) {
                res.push_back({cur_x, (c-
                    orig_a*cur_x)/orig_b});
            }
        }
        return {cnt, res};
    }
```

```
sign_b);
ll rx1=x;
shift_solution(x, y, a, b, -(miny-y)/a);
if (y<miny) shift_solution(x, y, a, b, -
    sign_a);
if (y>maxy) return {0, {}};
ll lx2=x;
shift_solution(x, y, a, b, -(maxy-y)/a);
if (y>maxy) shift_solution(x, y, a, b,
    sign_a);
ll rx2=x;
if (lx2>rx2) swap(lx2, rx2);
ll lx=max(lx1, lx2), rx=min(rx1, rx2);
if (lx>rx) return {0, {}};
ll cnt = (rx-lx)/abs(b)+1;
vector<pair<ll, ll>> res;
if (solves) {
    for (ll cur_x=lx; cur_x<=rx;
        cur_x+=abs(b)) {
        res.push_back({cur_x, (c-
            orig_a*cur_x)/orig_b});
    }
}
return {cnt, res};
}
```

2.9 Pollard Rho Factorization

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 - i {11, 19, 11}). Time: $O(n^{1/4})$, less for numbers with small factors.

```
namespace PollardRho {
    mt19937 rnd(chrono::steady_clock::
        now().time_since_epoch().count());
    const int P = 1e6 + 9;
    ll seq[P];
    int primes[P], spf[P];
    inline ll add_mod(ll x, ll y, ll m)
    {
        return (x += y) < m ? x : x - m;
    }
    inline ll mul_mod(ll x, ll y, ll m)
    {
        ll res = (ll)((__int128)x * y %
            m);
        return res;
    }
    inline ll pow_mod(ll x, ll n, ll m)
    {
        ll res = 1 % m;
        for (; n >= 1) {
            if (n & 1) res = mul_mod(res
                , x, m);
            x = mul_mod(x, x, m);
        }
        return res;
    }
    //  $O(it * (\log n)^3)$ , guaranteed
    // accurate for all 64-bit integers
    inline bool miller_rabin(ll n) {
        if (n <= 2 || (n & 1 ^ 1))
            return (n == 2);
        if (n < P) return spf[n] == n;
        ll c, d, s = 0, r = n - 1;
        for (; !(r & 1); r >= 1, s++)
            // Bound shifted to < 40 to
            // include prime base 37
            for (int i = 0; primes[i] < n &&
                primes[i] < 40; i++) {
                c = pow_mod(primes[i], r, n)
                    ;
                for (int j = 0; j < s; j++)
                    d = mul_mod(c, c, n);
                if (d == 1 && c != 1 &&
                    c != (n - 1)) return false;
                c = d;
            }
        if (c != 1) return false;
        return true;
    }
    void init() {
        int cnt = 0;
        for (int i = 2; i < P; i++) {
```

```

    if (!spf[i]) primes[cnt++] =
    spf[i] = i;
    for (int j = 0, k; (k = i *
    primes[j]) < P; j++) {
        spf[k] = primes[j];
        if (spf[i] == spf[k])
            break;
    }
}
// O(n^(1/4))
ll pollard_rho(ll n) {
    while (1) {
        ll x = rnd() % n, y = x, c =
        rnd() % n, u = 1, v, t = 0;
        ll *px = seq, *py = seq;
        while (1) {
            *py++ = y = add_mod(
            mul_mod(y, y, n), c, n);
            *py++ = y = add_mod(
            mul_mod(y, y, n), c, n);
            if ((x = *px++) == y)
                break;
            v = u;
            // Direct ternary avoids
            potential 32-bit truncation from
            global abs()
            u = mul_mod(u, (y > x ?
            y - x : x - y), n);
            if (!u) return __gcd(v,
            n);
            if (++t == 32) {
                t = 0;
                if ((u = __gcd(u, n)
                ) > 1 && u < n) return u;
            }
            if (t && (u = __gcd(u, n)) >
            1 && u < n) return u;
        }
    }
}
vector<ll> factorize(ll n) {
    if (n == 1) return vector<ll>();
    if (miller_rabin(n)) return
    vector<ll>{n};
    vector<ll> v, w;
    while (n > 1 && n < P) {
        v.push_back(spf[n]);
        n /= spf[n];
    }
    if (n >= P) {
        ll x = pollard_rho(n);
        v = factorize(x);
        w = factorize(n / x);
        v.insert(v.end(), w.begin(),
        w.end());
    }
    return v;
}
}

```

3 Dynamic Programming

3.1 SOS DP

```

const int B = 20;
int a[1 << B], f[1 << B], g[1 << B];
for (int i = 1; i <= n; i++) {
    cin >> a[i];
    f[a[i]]++;
    g[a[i]]++;
}
// sum over subsets
for (int i = 0; i < B; i++) {
    for (int mask = 0; mask < (1 << B);
    mask++) {
        if ((mask & (1 << i)) != 0) {
            f[mask] += f[mask ^ (1 << i)];
        }
    }
}
// sum over supersets
for (int i = 0; i < B; i++) {
    for (int mask = (1 << B) - 1; mask >=
    0; mask--) {
        if ((mask & (1 << i)) == 0) g[mask]
        += g[mask ^ (1 << i)];
    }
}
}

```

3.2 CHT

```

struct CHT {
    vector<ll> m, b;
    int ptr = 0;
    bool bad(int l1, int l2, int l3) {
        return 1.0 * (b[l3] - b[l1]) * (m[l1]
        - m[l2]) <=
            1.0 * (b[l2] - b[l1]) *
            (m[l1] - m[l3]); // (
            slope dec+query min), (slope inc+
            query max)
        return 1.0 * (b[l3] - b[l1]) * (m[l1]
        - m[l2]) >
            1.0 * (b[l2] - b[l1]) *
            (m[l1] - m[l3]); // (
            slope dec+query max), (slope inc+
            query min)
    }
    void add(ll _m, ll _b) {
        m.push_back(_m);
        b.push_back(_b);
        int s = m.size();
        while (s >= 3 && bad(s - 3, s - 2,
        s - 1)) {
            s--;
            m.erase(m.end() - 2);
            b.erase(b.end() - 2);
        }
    }
    ll f(int i, ll x) { return m[i] * x
    + b[i]; }
    // (slope dec+query min), (slope inc+
    query max) -> x increasing
    // (slope dec+query max), (slope inc+
    query min) -> x decreasing
    ll query(ll x) {
        if (ptr >= m.size()) ptr = m.size()
        - 1;
        while (ptr < m.size() - 1 && f(ptr
        + 1, x) < f(ptr, x)) ptr++;
        return f(ptr, x);
    }
    ll bs(int l, int r, ll x) {
        int mid = (l + r) / 2;
        if (mid + 1 < m.size() && f(mid +
        1, x) < f(mid, x))
            return bs(mid + 1, r, x); // >
        for max
        if (mid - 1 >= 0 && f(mid - 1, x)
        < f(mid, x))
            return bs(l, mid - 1, x); // >
        for max
        return f(mid, x);
    }
};

```

3.3 CHT Dynamic

```

struct Line {
    mutable int64_t m, b, p;
    bool operator<(const Line& o) const {
        return m < o.m; }
    bool operator<(int64_t x) const {
        return p < x; }
};
struct CHT : multiset<Line, less<>> {
    bool isMax;
    CHT(bool _isMax = true) { isMax =
    _isMax; }
    static int64_t div(int64_t a, int64_t
    b) {
        return a / b - ((a ^ b) < 0 && a % b
        );
    }
    bool isect(iterator x, iterator y) {
        if (y == end()) {
            x->p = LLONG_MAX;
            return false;
        }
        if (x->m == y->m) {
            x->p = (x->b > y->b ? LLONG_MAX :
            LLONG_MIN);
        } else {
            x->p = div(y->b - x->b, x->m - y->
            m);
        }
        return x->p >= y->p;
    }
};
pair<int64_t, int64_t> convert(int64_t

```

```

m, int64_t b) {
    if (isMax) return {m, b};
    return {-m, -b};
}
void add(int64_t m, int64_t b) {
    tie(m, b) = convert(m, b);
    auto z = insert({m, b, 0});
    auto y = z++;
    auto x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) {
        isect(x, y = erase(y));
    }
    while ((y = x) != begin() && (--x)->
    p >= y->p) {
        isect(x, erase(y));
    }
}
static int64_t eval(const Line& l,
int64_t x) {
    return (int64_t)((__int128)l.m * x +
    l.b);
}
int64_t query(int64_t x) {
    assert(!empty());
    auto l = *lower_bound(x);
    int64_t ans = eval(l, x);
    if (!isMax) ans = -ans;
    return ans;
}
};

```

4 Data Structure

4.1 Fenwick Tree

Description: Description: Computes partial sums $a[0] + a[1] + \dots + a[\text{pos} - 1]$, and updates single elements $a[i]$, taking the difference between the old and new value. **Time:** Both operations are $O(\log N)$

```

struct FT {
    std::vector<ll> s;
    FT(int n) : s(n + 1, 0) {}
    void update(int pos, ll dif) {
        for (; pos < s.size(); pos += pos &
        -pos) s[pos] += dif;
    }
    ll query(int pos) {
        ll res = 0;
        for (; pos > 0; pos -= pos & -pos)
            res += s[pos];
        return res;
    }
    ll query(int l, int r) {
        return query(r) - query(l - 1);
    }
    void update(int l, int r, ll dif) {
        update(l, dif);
        if (r + 1 < s.size()) update(r + 1,
        -dif);
    }
    int lower_bound(ll sum) {
        if (sum <= 0) return 0;
        int pos = 0;
        for (int pw = 1 << 25; pw >= 1) {
            if (pos + pw < s.size() && s[pos +
            pw] < sum) {
                pos += pw;
                sum -= s[pos];
            }
        }
        return pos + 1;
    }
};

```

4.2 Fenwick 2D (Compressed)

Description: Computes sums $a[i, j]$ for all $i < I, j < J$, and increases single elements $a[i, j]$. Requires that the elements to be updated are known in advance (call `fakeUpdate()` before `init()`). **Time:** $O(\log^2 N)$. (Use persistent segment trees for $O(\log N)$)

```

struct FT2 {
    vector<vector<ll>> ys;
    vector<FT> ft;

```

```

FT2(int limx) : ys(limx + 1) {}
void fakeUpdate(int x, int y) {
    for (; x < ys.size(); x += x & -x) {
        ys[x].push_back(y);
    }
}
void init() {
    ft.clear();
    ft.resize(ys.size(), FT(0));
    for (int i = 0; i < ys.size(); ++i) {
        sort(ys[i].begin(), ys[i].end());
        ys[i].erase(unique(ys[i].begin(),
            ys[i].end()), ys[i].end());
        ft[i] = FT(ys[i].size());
    }
}
int update(int x, int y) {
    return lower_bound(ys[x].begin(), ys
        [x].end(), y - ys[x].begin()) + 1;
}
int query(int x, int y) {
    return upper_bound(ys[x].begin(), ys
        [x].end(), y) - ys[x].begin();
}
void update(int x, int y, ll dif) {
    for (; x < ys.size(); x += x & -x)
        ft[x].update(update(x, y), dif);
}
ll query(int x, int y) {
    ll sum = 0;
    for (; x > 0; x -= x & -x) {
        sum += ft[x].query(query(x, y));
    }
    return sum;
}
ll query(ll x1, ll y1, ll x2, ll y2) {
    return (query(x2, y2) - query(x1-1,
        y2) -
        query(x2, y1-1) + query(x1
            -1, y1-1) +
            5ll*MOD) % MOD;
}
};

```

4.3 BIT 2D

```

template<typename T>
struct BIT2D {
    int n, m; vector<vector<T>>> t;
    BIT2D(int n, int m) : n(n), m(m), t(n+2,
        vector<T>(m+2, 0)) {}
    void update(int x, int y, T val) {
        for (int i = x; i <= n; i += (i & -i))
            for (int j = y; j <= m; j += (j & -j))
                t[i][j] += val;
    }
    void range_update(int x1, int y1, int x2,
        int y2, T val) {
        update(x1, y1, val);
        update(x1, y2+1, -val);
        update(x2+1, y1, -val);
        update(x2+1, y2+1, val);
    }
    T query(int x, int y) const {
        T sum = 0;
        for (int i = x; i > 0; i -= (i & -i))
            for (int j = y; j > 0; j -= (j & -j))
                sum += t[i][j];
        return sum;
    }
    T query(int x1, int y1, int x2, int y2)
        const {
        return query(x2, y2) - query(x1-1, y2) -
            query(x2, y1-1) + query(x1-1, y1-1);
    }
    void reset() {
        for (int i = 0; i <= n; ++i)
            fill(t[i].begin(), t[i].end(), 0);
    }
};

```

4.4 Segment Tree

```

template<typename T> // 1 based
struct ST {
    #define lc (nd << 1)
    #define rc ((nd << 1) | 1)
    vector<T> t;
    T identity = 0; // 0 for sum, 1e18
        for min etc;

```

```

ST(int n) { t.resize(4 * n, identity); }
inline T combine(T a, T b) { return a
    + b; }
inline void pull(int nd) { t[nd] =
    combine(t[lc], t[rc]); }
void build(int nd, int b, int e,
    vector<T>& a) {
    if (b == e) {
        t[nd] = a[b];
        return;
    }
    int mid = (b + e) >> 1;
    build(lc, b, mid, a);
    build(rc, mid + 1, e, a);
    pull(nd);
}
void upd(int nd, int b, int e, int i,
    T v) {
    if (b == e) {
        t[nd] = v; // For "add v", use t[
            nd] += v;
        return;
    }
    int mid = (b + e) >> 1;
    if (i <= mid) upd(lc, b, mid, i, v);
    else upd(rc, mid + 1, e, i, v);
    pull(nd);
}
T query(int nd, int b, int e, int i,
    int j) {
    if (i > e || b > j) return identity;
    if (i <= b && e <= j) return t[nd];
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, i,
        j), query(rc, mid + 1, e, i, j));
}
};

```

4.5 Lazy Propagation

```

struct ST {
    #define lc (n << 1)
    #define rc ((n << 1) | 1)
    // long long t[4 * N], lazy[4 * N];
    vector<ll> a, t, lazy;
    ST(vll& a) : a(a) {
        lazy.resize(4*a.size()+1), t.resize
            (4*a.size()+1);
    }
    ST(int n) {
        a.resize(n), lazy.resize(4*n), t.
            resize(4*n);
    }
    inline void push(int n, int b, int e)
        {
        if (lazy[n] == 0) return;
        t[n] = (ll) t[n] + (ll) lazy[n] * (
            ll) (e - b + 1);
        t[n] %= mod;
        if (b != e) {
            lazy[lc] = (ll) lazy[lc] + lazy[n]
                ;
            lazy[rc] = (ll) lazy[rc] + lazy[n]
                ;
            lazy[rc] %= mod, lazy[lc] %= mod;
        }
        lazy[n] = 0;
    }
    inline long long combine(long long a,
        long long b) {
        return (a + b) % mod;
    }
    inline void pull(int n) {
        t[n] = (t[lc] + t[rc]) % mod;
    }
    void build(int n, int b, int e) {
        lazy[n] = 0;
        if (b == e) {
            t[n] = a[b];
            return;
        }
        int mid = (b + e) >> 1;
        build(lc, b, mid);
        build(rc, mid + 1, e);
        pull(n);
    }
    void upd(int n, int b, int e, int i,

```

```

    int j, long long v) {
        push(n, b, e);
        if (j < b || e < i) return;
        if (i <= b && e <= j) {
            lazy[n] = v; //set lazy
            push(n, b, e);
            return;
        }
        int mid = (b + e) >> 1;
        upd(lc, b, mid, i, j, v);
        upd(rc, mid + 1, e, i, j, v);
        pull(n);
    }
    long long query(int n, int b, int e,
        int i, int j) {
        push(n, b, e);
        if (i > e || b > j) return 0; //
            return null
        if (i <= b && e <= j) return t[n];
        int mid = (b + e) >> 1;
        return (combine(query(lc, b, mid, i,
            j), query(rc, mid + 1, e, i, j)))
            % mod;
    }
};

```

4.6 Persistent Segment Tree

```

template<typename T>
struct PST {
    struct node {
        T val;
        int l, r;
    };
    int n;
    T identity = 0; // 0 for sum, INF for
        min etc
    vector<node> nodes;
    vector<int> version;
    T merge(T x, T y) { return x + y; }
    PST() {}
    PST(int sz) {
        n = sz;
        build(vector<T>(n, identity));
    }
    PST(vector<T>& a) {
        n = a.size();
        build(a);
    }
    int build(const vector<T>& a) {
        nodes.reserve(20 * n);
        int root = _build(0, n - 1, a);
        version.push_back(root);
        return version.size() - 1;
    }
    int _build(int l, int r, const vector<
        T>& a) {
        int root = nodes.size();
        nodes.push_back({});
        if (l == r) {
            nodes[root].val = a[l];
            return root;
        }
        int mid = (l + r) / 2;
        int ch = _build(l, mid, a);
        nodes[root].l = ch;
        ch = _build(mid + 1, r, a);
        nodes[root].r = ch;
        nodes[root].val = merge(nodes[nodes[
            root].l].val, nodes[nodes[root].r].
            val);
        return root;
    }
    int update(int i, T x, int v = -1) {
        if (v == -1) v = version.size() - 1;
        int root = _update(version[v], 0, n
            - 1, i, x);
        version.push_back(root);
        return version.size() - 1;
    }
    int _update(int u, int l, int r, int i
        , T x) {
        int root = nodes.size();
        int old = nodes[u];
        nodes.push_back(old);
        if (l == r) {
            nodes[root].val = x; // change =
                to += for add

```



```

    return root;
}
int mid = (l + r) / 2;
if (i <= mid) {
    int ch = _update(nodes[u].l, l,
        mid, i, x);
    nodes[root].l = ch;
} else {
    int ch = _update(nodes[u].r, mid +
        1, r, i, x);
    nodes[root].r = ch;
}
nodes[root].val = merge(nodes[nodes[
    root].l].val, nodes[nodes[root].r].
    val);
return root;
}
int update_batch(const vector<pair<int
    , T>>& up, int v = -1) {
    if (v == -1) v = version.size() - 1;
    if (up.empty()) { version.push_back(
        version[v]); return version.size()
        - 1; }
    int root = _update_batch(version[v],
        0, n - 1, up, 0, (int)up.size() -
        1);
    version.push_back(root);
    return version.size() - 1;
}
// Updates values up[x].first with
// delta up[x].second. Assumes that up
// is sorted.
int _update_batch(int u, int l, int r,
    const vector<pair<int, T>>& up,
    int ql, int qr) {
    if (ql > qr) return u;
    int root = nodes.size();
    nodes.push_back(nodes[u]);
    if (l == r) {
        for (int i = ql; i <= qr; ++i)
            nodes[root].val += up[i].second;
        return root;
    }
    int mid = (l + r) / 2;
    int sp = upper_bound(up.begin() + ql
        , up.begin() + qr + 1, mid,
        [](int m, const pair<int, T>& p) {
            return m < p.first; }) - up.begin
        ();
    nodes[root].l = _update_batch(nodes[
        u].l, l, mid, up, ql, sp - 1);
    nodes[root].r = _update_batch(nodes[
        u].r, mid + 1, r, up, sp, qr);
    nodes[root].val = merge(nodes[nodes[
        root].l].val, nodes[nodes[root].r].
        val);
    return root;
}
T get(int l, int r, int v) { return
    _get(version[v], 0, n - 1, l, r); }
T _get(int u, int tl, int tr, int l,
    int r) {
    if (l > r || r < tl) return
        identity;
    if (l <= tl && r >= tr) return nodes
        [u].val;
    int mid = (tl + tr) / 2;
    return merge(_get(nodes[u].l, tl,
        mid, l, r),
        _get(nodes[u].r, mid + 1, tr, l, r
        ));
}
// version is 1 indexed (0 is the
// initial before updates)
T get_kth(int ver_l, int ver_r, int k)
{
    return _get_kth(version[ver_l - 1],
        version[ver_r], 0, n - 1, k);
}
int _get_kth(int u, int v, int l, int
    r, int k) {
    if (l == r) return l;
    int mid = (l + r) / 2;
    T kl = nodes[nodes[v].l].val - nodes
        [nodes[u].l].val;
    if (kl >= k)
        return _get_kth(nodes[u].l, nodes[
            v].l, l, mid, k);

```

```

    else
        return _get_kth(nodes[u].r, nodes[
            v].r, mid + 1, r, k - kl);
}
};

4.7 Merge Sort Tree


---


// 0-based index
template <typename T>
struct MST {
    int n;
    vector<vector<T>> tree;
    MST(vector<T>& a) : n(a.size()) {
        tree.resize(4 * n);
        build(0, 0, n - 1, a);
    }
    void build(int u, int l, int r, vector
        <T>& a) {
        if (l == r) {
            tree[u].pb(a[l]);
            return;
        }
        int mid = (l + r) / 2;
        build(2 * u + 1, l, mid, a);
        build(2 * u + 2, mid + 1, r, a);
        tree[u].resize(r - l + 1);
        merge(all(tree[2 * u + 1]), all(tree
            [2 * u + 2]), tree[u].begin());
    }
    // Returns number of elements smaller
    // than x in range l, r
    int query(int l, int r, T x) { return
        get(0, 0, n - 1, l, r, x); }
    int get(int u, int tl, int tr, int l,
        int r, T x) {
        if (tl >= l && tr <= r)
            return lower_bound(all(tree[u]), x
            ) - tree[u].begin();
        if (tl > r || tr < l) return 0;
        int mid = (tl + tr) / 2;
        return get(2 * u + 1, tl, mid, l, r,
            x) +
            get(2 * u + 2, mid + 1, tr, l
            , r, x);
    }
};

4.8 Merge Sort Tree with Update


---


// 0 based indexing
template <typename T>
struct MST {
    int n;
    vector<ordered_set<pair<T, int>>> tree
        ;
    vector<T> a;
    MST(vector<T>& v) : a(v) {
        n = v.size();
        tree.resize(4 * n);
        build(0, 0, n - 1, a);
    }
    void merge(int u, int l, int r) {
        tree[u] = tree[l];
        for (auto x : tree[r]) tree[u].
            insert(x);
    }
    void build(int u, int l, int r, vector
        <T>& a) {
        if (l == r) {
            tree[u].insert({a[l], 1});
            return;
        }
        int mid = (l + r) / 2;
        build(2 * u + 1, l, mid, a);
        build(2 * u + 2, mid + 1, r, a);
        merge(u, 2 * u + 1, 2 * u + 2);
    }
    // Returns number of elements smaller
    // than x in range l, r
    int query(int l, int r, T x) { return
        get(0, 0, n - 1, l, r, x); }
    void update(int i, T x) {
        _update(0, 0, n - 1, i, x);
        a[i] = x;
    }
    int get(int u, int tl, int tr, int l,
        int r, T x) {

```

```

        if (tl >= l && tr <= r) return tree[
            u].order_of_key({x, 0});
        if (tl > r || tr < l) return 0;
        int mid = (tl + tr) / 2;
        return get(2 * u + 1, tl, mid, l, r,
            x) +
            get(2 * u + 2, mid + 1, tr, l
            , r, x);
    }
}
void _update(int u, int l, int r, int
    i, T x) {
    tree[u].erase({a[i], i});
    tree[u].insert({x, i});
    if (l == r) return;
    int mid = (l + r) / 2;
    if (i <= mid)
        _update(2 * u + 1, l, mid, i, x);
    else
        _update(2 * u + 2, mid + 1, r, i,
            x);
}
};

4.9 Treap


---


struct ImplicitTreap {
    struct Node { int l, r, sz, sum, val,
        priority, lazy; };
    vector<Node> tree;
    mt19937 rnd;
    ImplicitTreap(int max_nodes = 0) {
        rnd.seed(chrono::steady_clock::now()
            .time_since_epoch().count());
        if (max_nodes > 0) tree.reserve(
            max_nodes);
    }
    int getSize(int t) { return t == -1 ?
        0 : tree[t].sz; }
    int getSum(int t) { return t == -1 ? 0
        : tree[t].sum; }
    void applyLazy(int t, int add) {
        if (t == -1) return;
        tree[t].val += add; tree[t].sum +=
            tree[t].sz * add; tree[t].lazy +=
            add;
    }
    void pull(int t) {
        if (t == -1) return;
        tree[t].sz = 1 + getSize(tree[t].l)
            + getSize(tree[t].r);
        tree[t].sum = tree[t].val + getSum(
            tree[t].l) + getSum(tree[t].r);
    }
    void push(int t) {
        if (t == -1 || tree[t].lazy == 0)
            return;
        applyLazy(tree[t].l, tree[t].lazy);
        applyLazy(tree[t].r, tree[t].lazy);
        tree[t].lazy = 0;
    }
    int newNode(int val) {
        tree.push_back({-1, -1, 1, val, val,
            (int)rnd(), 0});
        return tree.size() - 1;
    }
    pair<int, int> split(int t, int k) {
        if (t == -1) return {-1, -1};
        push(t);
        if (getSize(tree[t].l) >= k) {
            auto res = split(tree[t].l, k);
            tree[t].l = res.second; pull(t);
            return {res.first, t};
        } else {
            auto res = split(tree[t].r, k -
                getSize(tree[t].l) - 1);
            tree[t].r = res.first; pull(t);
            return {t, res.second};
        }
    }
    int merge(int a, int b) {
        if (a == -1) return b;
        if (b == -1) return a;
        if (tree[a].priority > tree[b].
            priority) {
            push(a); tree[a].r = merge(tree[a
                ].r, b); pull(a);
            return a;
        } else {

```

```

    push(b); tree[b].l = merge(a, tree
[b].l); pull(b);
    return b;
}
}
int insert(int root, int pos, int val)
{
    int nn = newNode(val);
    auto [L, R] = split(root, pos);
    return merge(merge(L, nn), R);
}
int erase(int root, int pos) {
    auto [L, R] = split(root, pos);
    auto [p, q] = split(R, 1);
    return merge(L, q);
}
void show(int t) {
    if (t == -1) return;
    push(t); show(tree[t].l);
    cout << tree[t].val << " "; show(
tree[t].r);
}
};

```

4.10 Segment Tree 2D (1 based)

```

// USAGE
ST2D sg(NX, NY); // dimensions 1..NX,
1..NY
sg.upd (1, 1, NX, x, y, v);
sg.query (1, 1, NX, x1, x2, y1, y2);
struct ST2D {
    #define lc (nd << 1)
    #define rc ((nd << 1) | 1)
    #define lcy (ndy << 1)
    #define rcy ((ndy << 1) | 1)
    int NX, NY;
    vector<vector<long long>>> t;
    ST2D(int NX, int NY) : NX(NX), NY(NY),
        t(4 * NX, vector<long long>(4 * NY
        , 0)) {}
    inline long long combine(long long a,
        long long b) { return max(a, b); }
    inline void pull_y(int nd, int ndy) {
        t[nd][ndy] = combine(t[nd][lcy], t[
        nd][rcy]);
    }
    inline void pull_x(int nd, int ndy) {
        t[nd][ndy] = combine(t[lc][ndy], t[
        rc][ndy]);
    }
    // -----
    // Inner seg tree on y-axis
    // -----
    void upd_y(int nd, int ndy, int by,
        int ey, int j, long long v, bool
        leaf_x) {
        if (by == ey) {
            if (leaf_x) t[nd][ndy] = v;
            else t[nd][ndy] = combine(t
            [lc][ndy], t[rc][ndy]);
            return;
        }
        int mid = (by + ey) >> 1;
        if (j <= mid) upd_y(nd, lcy, by, mid
        , j, v, leaf_x);
        else upd_y(nd, rcy, mid+1,
        ey, j, v, leaf_x);
        pull_y(nd, ndy);
    }
    long long query_y(int nd, int ndy, int
        by, int ey, int y1, int y2) {
        if (y1 > ey || by > y2) return 0;
        if (y1 <= by && ey <= y2) return t[
        nd][ndy];
        int mid = (by + ey) >> 1;
        return combine(query_y(nd, lcy, by,
        mid, y1, y2),
            query_y(nd, rcy, mid
        +1, ey, y1, y2));
    }
    // -----
    // Outer seg tree on x-axis
    // -----
    void upd(int nd, int bx, int ex, int i
        , int j, long long v) {
        if (bx == ex) {
            upd_y(nd, 1, 1, NY, j, v, true);

```

```

        return;
    }
    int mid = (bx + ex) >> 1;
    if (i <= mid) upd(lc, bx, mid, i, j,
        v);
    else upd(rc, mid+1, ex, i,
        j, v);
    upd_y(nd, 1, 1, NY, j, v, false);
    // pull x into inner nodes
}
long long query(int nd, int bx, int ex
    , int x1, int x2, int y1, int y2) {
    if (x1 > ex || bx > x2) return 0;
    if (x1 <= bx && ex <= x2) return
        query_y(nd, 1, 1, NY, y1, y2);
    int mid = (bx + ex) >> 1;
    return combine(query(lc, bx, mid, x1
        , x2, y1, y2),
        query(rc, mid+1, ex,
        x1, x2, y1, y2));
}
};

```

4.11 Segment Tree 2D (Index compression)

```

// USAGE
ST2D sg(n);
// Phase 1: register all (x, y) points
// that will ever be updated
for (auto& [x, y, v] : updates)
    seg.collect(1, 1, n, x, y);
// Phase 2: compress and allocate inner
// trees
seg.build(1, 1, n);
// Updates and queries
seg.upd (1, 1, n, x, y, v);
seg.query (1, 1, n, x1, x2, y1, y2);
//
// //////////////////////////////////////
void compress(vector<int>& a) {
    sort(all(a));
    a.erase(unique(all(a)), a.end());
}
int ind(vi& a, int val) {
    return lower_bound(all(a), val) - a.
        begin();
}
struct ST2D {
    #define lc (nd << 1)
    #define rc ((nd << 1) | 1)
    int n;
    vector<vi> ys; // compressed y-
        coords per x-node
    vector<vi> t; // inner seg tree
        per x-node
    ST2D(int n) : n(n), ys(4 * n), t(4 * n
        ) {}
    void collect(int nd, int b, int e, int
        x, int y) {
        ys[nd].pb(y);
        if (b == e) return;
        int mid = (b + e) >> 1;
        if (x <= mid) collect(lc, b, mid, x,
            y);
        else collect(rc, mid + 1, e
            , x, y);
    }
    void build(int nd, int b, int e) {
        compress(ys[nd]);
        t[nd].assign(ys[nd].sz() * 2, 0);
        if (b == e) return;
        int mid = (b + e) >> 1;
        build(lc, b, mid);
        build(rc, mid + 1, e);
    }
    inline long long combine(long long a,
        long long b) { return max(a, b); }
    void upd_y(int nd, int y, long long v)
        {
        int m = ys[nd].sz();
        int i = ind(ys[nd], y) + m;
        t[nd][i] = combine(t[nd][i], v);
        for (i >= 1; i >= 1; i >= 1)
            t[nd][i] = combine(t[nd][i*2], t[
            nd][i*2+1]);
    }
}

```

```

long long query_y(int nd, int y1, int
    y2) {
    if (y1 > y2) return 0;
    int m = ys[nd].sz();
    int l = ind(ys[nd], y1) + m;
    int r = (int)(upper_bound(all(ys[nd
        ]), y2) - ys[nd].begin() - 1) + m;
    ll mx = 0;
    while (l <= r) {
        if (l%2 == 1) mx = combine(mx, t
            [nd][l++]);
        if (r%2 == 0) mx = combine(mx, t
            [nd][r--]);
        l >>= 1; r >>= 1;
    }
    return mx;
}
void upd(int nd, int b, int e, int x,
    int y, long long v) {
    upd_y(nd, y, v);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) upd(lc, b, mid, x, y,
        v);
    else upd(rc, mid + 1, e, x,
        y, v);
}
long long query(int nd, int b, int e,
    int x1, int x2, int y1, int y2) {
    if (x1 > x2 || y1 > y2) return 0;
    // empty range (e.g. a[i].f == 1)
    if (x1 > e || b > x2) return 0;
    if (x1 <= b && e <= x2) return
        query_y(nd, y1, y2);
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, x1,
        x2, y1, y2),
        query(rc, mid+1, e,
        x1, x2, y1, y2));
}
};

```

4.12 DSU

```

struct DSU {
    vector<int> par, sz;
    DSU(){}
    DSU(int n){
        init(n);
    }
    void init(int n){
        par.resize(n);
        iota(par.begin(), par.end(), 0);
        sz.assign(n, 1);
    }
    int find(int v){
        while(v != par[v]){
            v = par[v] = par[par[v]];
        }
        return v;
    }
    bool merge(int a, int b){
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        par[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

```

4.13 SQRT Decomposition

```

struct SqrtDecomp {
    int n, sz;
    vector<ll> a;
    vector<map<int, int>>> b;
    SqrtDecomp(int n, vi& arr) : n(n) {
        sz = sqrt(n) + 1;
        /*...*/
    }
    void upd(int i, ll v) {
        /*...*/
    }
    ll combine(ll s, ll elem, int v) {
        /*...*/
    }
    ll combineBlock(ll s, map<int, int>&
        blk, int v) {

```

```

    /*...*/
}
ll query(int l, int r, int v) {
    ll s = 0;
    r--;
    if(l > r) {
        return 0;
    }
    int bl = l/sz, br = r/sz;
    if (bl == br) {
        for (int i = l; i <= r; i++)
            s = combine(s, a[i], v);
    } else {
        for (int i = l; i < (bl+1)*sz; i++) s = combine(s, a[i], v);
        for (int i = bl+1; i < br; i++) s = combineBlock(s, b[i], v);
        for (int i = br*sz; i <= r; i++) s = combine(s, a[i], v);
    }
    return s;
}
};

```

4.14 MOs Algorithm

```

struct MO{
    int n,q,B,res; struct query{ int l,r, id; };
    vector<query> Q; vector<int> ans,cnt;
    MO(int n, int q): n(n), q(q){
        res=0;
        B=sqrt(n)+1;
        ans.resize(q+1);
        cnt.resize(n+10);
    }
    void add_query(int l, int r, int id){
        Q.push_back({l,r,id});
    }
    void add_left(int id,const vector<int> &a){}
    void add_right(int id,const vector<int> &a){}
    void rem_left(int id,const vector<int> &a){}
    void rem_right(int id,const vector<int> &a){}
    void get_ans(const vector<int> &a){
        sort(Q.begin(),Q.end(), [&](auto &x, auto &y){
            if(x.l/B == y.l/B){
                return ((x.l/B)&1) ? x.r>y.r : x.r<y.r;
            }
            return x.l/B < y.l/B;
        });
        int L=1,R=0;
        for(auto &[l,r,id]: Q){
            while(L>l) add_left(--L,a);
            while(R<r) add_right(++R,a);
            while(L<l) rem_left(L++,a);
            while(R>r) rem_right(R--,a);
            ans[id]=res;
        }
    }
};

```

4.15 MOs with updates

```

struct mos_update {
    ll a[100005];
    ll ans;
    mos_update() { /* ... */ }
    void add(int ind, int end, int L, int R) { /* ... */ }
    void del(int ind, int end, int L, int R) { /* ... */ }
    void apply(int pos, int val, int L, int R) {
        if (L <= pos && pos <= R) {
            del(pos, -1, L, R); a[pos] = val;
            add(pos, -1, L, R);
        } else {
            a[pos] = val;
        }
    }
    vector<ll> mo(vector<vector<int>>& Q, vector<vector<int>>& U, int n) {
        int L = 1, R = 0, T = 0;
        ans = 0;
    }
};

```

```

int blk = max(1, (int)pow(Q.size(), 0.666));
vector<int> s(Q.size());
vector<ll> res(Q.size());
iota(s.begin(), s.end(), 0);
sort(s.begin(), s.end(), [&](int i, int j) {
    int li = Q[i][0] / blk, lj = Q[j][0] / blk;
    if (li != lj) return li < lj;
    int ri = Q[i][1] / blk, rj = Q[j][1] / blk;
    if (ri != rj) return (li & 1) ? ri < rj : ri > rj;
    return (ri & 1) ? Q[i][2] < Q[j][2] : Q[i][2] > Q[j][2];
});
for (int qi : s) {
    int ql = Q[qi][0], qr = Q[qi][1], qt = Q[qi][2];
    while (T < qt) { apply(U[T][0], U[T][2], L, R); T++; }
    while (T > qt) { --T; apply(U[T][0], U[T][1], L, R); }
    while (L > ql) { --L; add(L, 0, L, R); }
    while (R < qr) { ++R; add(R, 1, L, R); }
    while (L < ql) { del(L, 0, L, R); L++; }
    while (R > qr) { del(R, 1, L, R); R--; }
    res[qi] = ans;
}
return res;
};

```

4.16 Treap

```

struct ImplicitTreap {
    struct Node { int l, r, sz, sum, val, priority, lazy; };
    vector<Node> tree;
    mt19937 rnd;
    ImplicitTreap(int max_nodes = 0) {
        rnd.seed(chrono::steady_clock::now().time_since_epoch().count());
        if (max_nodes > 0) tree.reserve(max_nodes);
    }
    int getSize(int t) { return t == -1 ? 0 : tree[t].sz; }
    int getSum(int t) { return t == -1 ? 0 : tree[t].sum; }
    void applyLazy(int t, int add) {
        if (t == -1) return;
        tree[t].val += add; tree[t].sum += tree[t].sz * add; tree[t].lazy += add;
    }
    void pull(int t) {
        if (t == -1) return;
        tree[t].sz = 1 + getSize(tree[t].l) + getSize(tree[t].r);
        tree[t].sum = tree[t].val + getSum(tree[t].l) + getSum(tree[t].r);
    }
    void push(int t) {
        if (t == -1 || tree[t].lazy == 0) return;
        applyLazy(tree[t].l, tree[t].lazy);
        applyLazy(tree[t].r, tree[t].lazy);
        tree[t].lazy = 0;
    }
    int newNode(int val) {
        tree.push_back({-1, -1, 1, val, val, (int)rnd(), 0});
        return tree.size() - 1;
    }
    pair<int, int> split(int t, int k) {
        if (t == -1) return {-1, -1};
        push(t);
        if (getSize(tree[t].l) >= k) {
            auto res = split(tree[t].l, k);
            tree[t].l = res.second; pull(t);
            return {res.first, t};
        } else {

```

```

            auto res = split(tree[t].r, k - getSize(tree[t].l) - 1);
            tree[t].r = res.first; pull(t);
            return {t, res.second};
        }
    }
    int merge(int a, int b) {
        if (a == -1) return b;
        if (b == -1) return a;
        if (tree[a].priority > tree[b].priority) {
            push(a); tree[a].r = merge(tree[a].r, b); pull(a);
            return a;
        } else {
            push(b); tree[b].l = merge(a, tree[b].l); pull(b);
            return b;
        }
    }
    int insert(int root, int pos, int val) {
        auto nn = newNode(val);
        auto [L, R] = split(root, pos);
        return merge(merge(L, nn), R);
    }
    int erase(int root, int pos) {
        auto [L, R] = split(root, pos);
        auto [p, q] = split(R, 1);
        return merge(L, q);
    }
    void show(int t) {
        if (t == -1) return;
        push(t); show(tree[t].l);
        cout << tree[t].val << " "; show(tree[t].r);
    }
};

```

4.17 Wavelet Tree

```

struct wavelet_tree {
    int lo, hi;
    wavelet_tree *l, *r;
    vector<int> b, c;
    wavelet_tree() {
        lo = hi = 0;
        l = r = NULL;
    }
    void init(int *from, int *to, int x, int y) {
        lo = x, hi = y;
        if (from >= to) return;
        int mid = (lo + hi) >> 1;
        auto f = [mid](int x) { return x <= mid; };
        b.reserve(to - from + 1);
        c.reserve(to - from + 1);
        b.push_back(0);
        c.push_back(0);
        for (auto it = from; it != to; ++it) {
            b.push_back(b.back() + f(*it));
            c.push_back(c.back() + (*it));
        }
        if (hi == lo) return;
        auto pivot = stable_partition(from, to, f);
        l = new wavelet_tree(); l->init(from, pivot, lo, mid);
        r = new wavelet_tree(); r->init(pivot, to, mid + 1, hi);
    }
    int kth(int l, int r, int k) {
        if (l > r) return 0;
        if (lo == hi) return lo;
        int inLeft = b[r] - b[l - 1], lb = b[l - 1], rb = b[r];
        if (k <= inLeft) return this->l->kth(lb + 1, rb, k);
        return this->r->kth(1 - lb, r - rb, k - inLeft);
    }
    int LTE(int l, int r, int k) {
        if (l > r || k < lo) return 0;
        if (hi <= k) return r - l + 1;
    }
};

```

```

    int lb = b[l - 1], rb = b[r];
    return this->l->LTE(lb + 1, rb,
k) + this->r->LTE(l - lb, r - rb, k
);
}
int count(int l, int r, int k) {
    if (l > r || k < lo || k > hi)
return 0;
    if (lo == hi) return r - l + 1;
    int lb = b[l - 1], rb = b[r],
mid = (lo + hi) >> 1;
    if (k <= mid) return this->l->
count(lb + 1, rb, k);
    return this->r->count(l - lb, r
- rb, k);
}
int sum(int l, int r, int k) {
    if (l > r || k < lo) return 0;
    if (hi <= k) return c[r] - c[l -
1];
    int lb = b[l - 1], rb = b[r];
    return this->l->sum(lb + 1, rb,
k) + this->r->sum(l - lb, r - rb, k
);
}
~wavelet_tree() { delete l; delete r
; }
};
wavelet_tree t;

```

4.18 Sparse Table (RMQ)

```

template<typename T>
struct SparseTable {
    const int N = 1e5 + 9;
    vector<vector<T>> t;
    vector<T> a;
    T combine(T x, T y) { return min(x, y)
; }
    void build(int n) {
        t.assign(n + 1, vector<T>(18));
        for(int i = 1; i <= n; ++i) t[i][0]
= a[i];
        for(int k = 1; k < 18; ++k)
            for(int i = 1; i + (1 << k) - 1 <=
n; ++i)
                t[i][k] = combine(t[i][k-1], t[i
+ (1 << (k-1))][k-1]);
    }
    T query(int l, int r) {
        int k = 31 - __builtin_clz(r - l +
1);
        return combine(t[l][k], t[r - (1 <<
k) + 1][k]);
    }
};

```

4.19 Prefix Sum 2D

```

template<typename T>
struct PrefixSum2D {
    vector<vector<T>> p;
    int rows, cols;
    T combine(T x, T y) { return x + y; }
    void build(vector<vector<T>>& a) {
        rows = a.size(); cols = a[0].size();
        p.assign(rows + 1, vector<T>(cols +
1, 0));
        for(int i = 1; i <= rows; ++i)
            for(int j = 1; j <= cols; ++j)
                p[i][j] = a[i-1][j-1] + p[i-1][j
] + p[i][j-1] - p[i-1][j-1];
    }
    // query sum over [r1,c1]..[r2,c2] (1-
indexed)
    T query(int r1, int c1, int r2, int c2
) {
        return p[r2][c2] - p[r1-1][c2] - p[
r2][c1-1] + p[r1-1][c1-1];
    }
};

```

4.20 Prefix Sum 3D

```

template<typename T>
struct PrefixSum3D {
    vector<vector<vector<T>>> p;
    int X, Y, Z;
    T combine(T x, T y) { return x + y; }

```

```

void build(vector<vector<vector<T>>>&
a) {
    X = a.size(); Y = a[0].size(); Z = a
[0][0].size();
    p.assign(X+1, vector<vector<T>>(Y+1,
vector<T>(Z+1, 0)));
    for(int i = 1; i <= X; ++i)
        for(int j = 1; j <= Y; ++j)
            for(int k = 1; k <= Z; ++k)
                p[i][j][k] = a[i-1][j-1][k-1]
+ p[i-1][j][k] + p[i][j-1][k
] + p[i][j][k-1]
- p[i-1][j-1][k] - p[i-1][j
][k-1] - p[i][j-1][k-1]
+ p[i-1][j-1][k-1];
    }
    // query sum over [x1,y1,z1]..[x2,y2,
z2] (1-indexed)
    T query(int x1, int y1, int z1, int x2
, int y2, int z2) {
        return p[x2][y2][z2]
- p[x1-1][y2][z2] - p[x2][y1-1][z2
] - p[x2][y2][z1-1]
+ p[x1-1][y1-1][z2] + p[x1-1][y2][
z1-1] + p[x2][y1-1][z1-1]
- p[x1-1][y1-1][z1-1];
    }
};

```

5 Graph

5.1 Dijkstra

```

struct Node {
    int u;
    ll dist;
    bool operator<(const Node& v) const {
        return dist > v.dist; }
};
vector<ll> dijkstra(int start, vector<
VPI>& adj) {
    vector<ll> dis(adj.size(), INFL);
    priority_queue<Node> q;
    dis[start] = 0;
    q.push({start, 0});
    while (!q.empty()) {
        Node node = q.top();
        int u = node.u;
        q.pop();
        if (dis[u] < node.dist) continue;
        for (auto e : adj[u]) {
            if (node.dist + e.ss < dis[e.ff])
            {
                dis[e.ff] = node.dist + e.ss;
                q.push({e.ff, dis[e.ff]});
            }
        }
    }
    return dis;
}

```

5.2 Bellmanford

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get $\text{dist} = \text{inf}$; nodes reachable through negative-weight cycles get $\text{dist} = -\text{inf}$. Assumes $V^2 \max |w| < \infty$ to prevent overflow.
Time: $O(VE)$

```

const ll inf = LLONG_MAX;
struct Ed { int a, b, w; ll s() { return
a < b ? a : -a; } };
struct Node { ll dist = inf; int prev =
-1; };
void bellmanFord(vector<Node>& nodes,
vector<Ed>& eds, int s) {
    nodes[s].dist = 0;
    sort(all(eds), [](Ed a, Ed b) { return
a.s() < b.s(); });
    int lim = nodes.size() / 2 + 2; //
/3+100 with shuffled vertices
    for(int i = 0; i < lim; i++) for (Ed
ed : eds) {
        Node cur = nodes[ed.a], &dest =
nodes[ed.b];
        if (abs(cur.dist) == inf) continue;
        ll d = cur.dist + ed.w;

```

```

    if (d < dest.dist) {
        dest.prev = ed.a;
        dest.dist = (i < lim-1 ? d : -inf)
;
    }
}
for(int i = 0; i < lim; i++) for (Ed e
: eds) {
    if (nodes[e.a].dist == -inf)
        nodes[e.b].dist = -inf;
}
}

```

5.3 BCC, Bridge, AP

Description: Finds biconnected components, bridges, and articulation points in a simple undirected graph.

Usage: Build graph using `g[u].push_back({v, edge_id})` and call `get.bcc(n)`. The vector `bccs` stores each biconnected component as a list of edge IDs. `bridges` stores all bridge edge IDs, and `is_art[u]` is true if node u is an articulation point. Graph must be **cleared** between test cases.

Time: $O(V + E)$

```

// g[u] = {{neighbor, edge_id}, ...}
vector<vector<pair<int,int>>> g;
vector<int> tin, st, bridges;
vector<bool> is_art;
vector<vector<int>> bccs;
int timer;
int dfs(int u, int pe) {
    int me = tin[u] = ++timer, top = me,
ch = 0;
    for (auto [v, id] : g[u]) if (id != pe
) {
        if (tin[v]) {
            top = min(top, tin[v]);
            if (tin[v] < me) st.push_back(id);
        } else {
            ch++;
            int si = st.size();
            int up = dfs(v, id);
            top = min(top, up);
            if (up >= me) {
                if (pe != -1) is_art[u] = true;
                if (up == me) {
                    st.push_back(id);
                    bccs.push_back({st.begin() +
si, st.end()});
                    st.resize(si);
                } else {
                    bridges.push_back(id);
                }
            } else {
                st.push_back(id);
            }
        }
    }
    if (pe == -1 && ch > 1) is_art[u] =
true;
    return top;
}

```

```

void get_bcc(int n) {
    tin.assign(n + 1, 0); is_art.assign(n
+ 1, false);
    st.clear(); bccs.clear(); bridges.
clear(); timer = 0;
    for (int i = 1; i <= n; i++) if (!tin[
i]) dfs(i, -1);
}

```

5.4 2CC

```

struct TECC{
    using Edge=array<int,2>;
    int n,cnt_comp;
    vector<vector<int>> G,T; vector<bool>
vis;
    vector<int> tin,low,comp; vector<Edge>
bridges;
    void low_dfs(int u, int par, int& cur)
    {
        vis[u]=true; tin[u]=cur++; low[u]=n
+1;
        bool usedPar=false;
        for(auto v: G[u]){
            if(vis[v]){
                if(v!=par || usedPar){

```



```

        low[u]=min(low[u],tin[v]);
    }else{
        usedPar=true;
    }
    continue;
}
low_dfs(v,u,cur);
low[u]=min(low[u],low[v]);
}
void build_dfs(int u, int cid){
    comp[u]=cid;
    for(auto v: G[u]){
        if(comp[v]!=-1) continue;
        if(tin[u]<low[v]){
            bridges.push_back({u,v});
            cnt_comp++;
            build_dfs(v,cnt_comp);
        }else{
            build_dfs(v,cid);
        }
    }
}
TECC(int _n,const vector<vector<int>>&
    g){
    n=_n; G=g; cnt_comp=1;
    tin.resize(n+1);
    low.resize(n+1);
    vis.assign(n+1,false);
    comp.assign(n+1,-1);
    int cur=1;
    for(int i=1;i<=n;i++){
        if(vis[i]) continue;
        low_dfs(i,-1,cur);
        build_dfs(i,cnt_comp);
        cnt_comp++;
    }
}
void build_tree(){
    T.assign(cnt_comp,{});
    for(auto &e: bridges){
        int u=comp[e[0]],v=comp[e[1]];
        T[u].push_back(v);
        T[v].push_back(u);
    }
}
};

```

5.5 LCA

```

struct LCA {
    int step = 0, k = 0;
    vector<int> vis_start, vis_end;
    vector<vector<int>> st;
    vector<vector<int>>& tree;
    LCA(vector<vector<int>>& tree) : tree(
        tree) { init(tree.size() - 1); }
    void build_st(int u, int p) {
        st[u][0] = p;
        for (int i = 1; i <= k; ++i) st[u][i]
            = st[st[u][i - 1]][i - 1];
    }
    void dfs(int u, int p) {
        vis_start[u] = step++;
        build_st(u, p);
        for (auto v : tree[u]) {
            if (v != p) dfs(v, u);
        }
        vis_end[u] = step++;
    }
    bool is_ancestor(int a, int b) {
        return vis_start[a] < vis_start[b]
            && vis_end[a] > vis_end[b];
    }
    int get(int a, int b) {
        if (a == b) return a;
        if (is_ancestor(a, b)) return a;
        if (is_ancestor(b, a)) return b;
        for (int i = k; i >= 0; --i) {
            if (!is_ancestor(st[a][i], b)) a =
                st[a][i];
        }
        return st[a][0];
    }
    void init(int n) {
        vis_start.assign(n + 1, 0);
        vis_end.assign(n + 1, 0);
        k = __lg(2 * n - 1);
    }
}

```

```

    st.assign(n + 1, vector<int>(k + 1))
    ;
    dfs(1, 1);
}
// Extra
int get_ancestor(int u, int n) {
    for (int i = 0; n > 0; ++i) {
        if (n & (1 << i)) {
            u = st[u][i];
            n -= (1 << i);
        }
    }
    return u;
}
};

```

5.6 SCC

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: `scc(graph, (vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components. $O(E+V)$

```

vector<int> val, comp, z;
int Time, ncomps;
vector<vector<int>> sccs;
int dfs(int j, vector<vector<int>>& g) {
    int low = val[j] = ++Time, x; z.
        push_back(j);
    for (int e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ? dfs(e,
            g));
    if (low == val[j]) {
        sccs.push_back({});
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            sccs.back().push_back(x);
        } while (x != j);
        ncomps++;
    }
    return val[j] = low;
}
void get_sccs(vector<vector<int>>& g) {
    int n = g.size();
    val.assign(n, 0); comp.assign(n, -1)
    ;
    z.clear(); sccs.clear();
    Time = ncomps = 0;
    for (int i = 0; i < n; ++i) if (comp
        [i] < 0) dfs(i, g);
}

```

5.7 Kosaraju

```

class Kosaraju{
public:
    int n;
    vector<vector<int32_t>> g, rg;
    vector<int32_t> comp,order,sz; vector<
        bool> vis;
    void dfs1(int u) {
        vis[u] = true;
        for (int v : g[u]) {
            if(!vis[v]) dfs1(v);
        }
        order.push_back(u);
    }
    void dfs2(int u, int c) {
        comp[u] = c; sz[comp[u]]++;
        for (auto& v : rg[u]) {
            if(comp[v] == -1) dfs2(v, c);
        }
    }
    Kosaraju(int n) : n(n) {
        g.assign(n + 1, {});
        rg.assign(n + 1, {});
        sz.assign(n + 1, 0);
    }
    inline void add_edge(int u, int v) {
        g[u].push_back(v);
        rg[v].push_back(u);
    }
}

```

```

int build() {
    vis.assign(n + 1, false);
    order.clear();
    for (int i = 1; i <= n; i++) {
        if (!vis[i]) dfs1(i);
    }
    comp.assign(n + 1, -1);
    reverse(order.begin(), order.end());
    int scc = 0;
    for (int u : order) {
        if (comp[u] == -1) {
            dfs2(u, ++scc);
        }
    }
    return scc;
}
vector<vector<int32_t>> build_dag(int
    scc) {
    vector<vector<int32_t>> dag(scc + 1)
    ;
    for (int u = 1; u <= n; u++) {
        for (int v : g[u]) {
            if (comp[u] != comp[v]){
                dag[comp[u]].push_back(comp[v]
                );
            }
        }
    }
    return dag;
}
};

```

5.8 HLD

```

vector<int> heavy, chain, label, par, sz
    , dep;
int timer;
void dfsz(int u, int p, vector<vector<
    int>>& tree) {
    sz[u] = 1;
    par[u] = p;
    dep[u] = dep[p] + 1;
    int mx = -1;
    for (int v : tree[u]) {
        if (v == p) continue;
        dfsz(v, u, tree);
        sz[u] += sz[v];
        if (sz[v] > mx) mx = sz[v], heavy[u]
            = v;
    }
}
void dfshld(int u, int p, vector<vector<
    int>>& tree, int head) {
    label[u] = timer++;
    chain[u] = head;
    if (heavy[u] != -1) dfshld(heavy[u], u
        , tree, head);
    for (int v : tree[u]) {
        if (v == p || v == heavy[u])
            continue;
        dfshld(v, u, tree, v);
    }
}
void inithld(vector<vector<int>>& tree)
    {
    int n = tree.size();
    heavy = vector<int>(n, -1);
    chain = vector<int>(n);
    label = vector<int>(n);
    sz = vector<int>(n);
    par = vector<int>(n);
    dep = vector<int>(n);
    timer = 1;
    dfsz(1, 0, tree);
    dfshld(1, 0, tree, 1);
}
// For path update, you can use the
// query function itself, change
// query with update
ll query(int u, int v, /*, fenwick/
    segtree/anything& ds*/) {
    ll ret = 0; // init value
    for (; chain[u] != chain[v]; v = par[
        chain[v]]) {
        if (dep[chain[u]] > dep[chain[v]])
            swap(u, v);
        // ret += ds.query(label[chain[v]],
            label[u]);
    }
}

```

```

}
if (dep[u] > dep[v]) swap(u, v);
// lca is u
// For EDGES, do label[u]+1
// ret = ds.query(label[u], label[v])
return ret;
}
ll querySubtree(int u, /*, fenwick/
    segtree/anything& ds */) {
    // For EDGES, do label[u]+1
    return ds.query(label[u], label[u]
        +sz[u]-1)
}
// Point Updating u : ds.update(label[u]
    ...)
```

5.9 Centroid Decomposition

```

struct CD {
    vector<vector<int>>& tree;
    vector<int> par, sz;
    vector<bool> dead;
    CD(vector<vector<int>>& tree) : tree(
        tree) {
        par.resize(tree.size());
        sz.resize(tree.size());
        dead.resize(tree.size());
        build(1, 0);
    }
    void build(int u, int p) {
        dfs(u, p);
        u = dfs(u, p, sz[u]);
        dead[u] = true;
        par[u] = p;
        // u is centroid, process here...
        for (int v : tree[u]) {
            if (!dead[v]) build(v, u);
        }
    }
    void dfs(int u, int p) {
        sz[u] = 1;
        for (int v : tree[u]) {
            if (v == p || dead[v]) continue;
            dfs(v, u);
            sz[u] += sz[v];
        }
    }
    int dfs(int u, int p, int n) {
        for (int v : tree[u]) {
            if (v == p || dead[v]) continue;
            if (sz[v] > n / 2) return dfs(v, u, n);
        }
        return u;
    }
    int operator[](int i) { return par[i];
    }
};
```

5.10 Arpa's Trick

```

const int N = 1e5 + 9;
vector<int> g[N];
int ans[N], col[N], sz[N], cnt[N];
bool big[N];
void dfs(int u, int p) {
    sz[u] = 1;
    for (auto v : g[u]) {
        if (v == p) continue;
        dfs(v, u);
        sz[u] += sz[v];
    }
}
void add(int u, int p, int x) {
    cnt[col[u]] += x;
    for (auto v : g[u]) {
        if (v == p || big[v] == 1)
            continue;
        add(v, u, x);
    }
}
void dsu(int u, int p, bool keep) {
    int bigchild = -1, mx = -1;
    for (auto v : g[u]) {
        if (v == p) continue;
        if (sz[v] > mx) mx = sz[v],
            bigchild = v;
    }
}
```

```

for (auto v : g[u]) {
    if (v == p || v == bigchild)
        continue;
    dsu(v, u, 0);
}
if (bigchild != -1) dsu(bigchild, u,
    1), big[bigchild] = 1;
add(u, p, 1);
ans[u] = cnt[u];
if (bigchild != -1) big[bigchild] =
    0;
if (keep == 0) add(u, p, -1);
}
```

6 Flow and Matching

6.1 Min Cost Max Flow

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $O(FE \log V)$ where F is max flow; $O(VE)$ for setpi.

```

const ll INF = numeric_limits<ll>::max()
    / 4;
struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;
    MCMF(int N) : N(N), ed(N), seen(N),
        dist(N), pi(N), par(N) {}
    void addEdge(int from, int to, ll cap,
        ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{from, to, sz
            (ed[to]), cap, cost, 0});
        ed[to].push_back(edge{to, from, sz(
            ed[from]) - 1, 0, -cost, 0});
    }
    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0;
        ll di;
        __gnu_pbds::priority_queue<pair<ll,
            int>> q;
        vector<decltype(q)::point_iterator>
            its(N);
        q.push({0, s});
        while (!q.empty()) {
            s = q.top().second;
            q.pop();
            seen[s] = 1;
            di = dist[s] + pi[s];
            for (edge& e : ed[s])
                if (!seen[e.to]) {
                    ll val = di - pi[e.to] + e.
                        cost;
                    if (e.cap - e.flow > 0 && val
                        < dist[e.to]) {
                        dist[e.to] = val;
                        par[e.to] = &e;
                        if (its[e.to] == q.end())
                            its[e.to] = q.push({-dist[
                                e.to], e.to});
                        else
                            q.modify(its[e.to], {-dist
                                [e.to], e.to});
                    }
                }
            rep(i, 0, N) pi[i] = min(pi[i] +
                dist[i], INF);
        }
    }
    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge* x = par[t]; x; x = par[
                x->from])
```

```

            fl = min(fl, x->cap - x->flow);
            totflow += fl;
            for (edge* x = par[t]; x; x = par[
                x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        rep(i, 0, N) for (edge& e : ed[i])
            totcost += e.cost * e.flow;
        return {totflow, totcost / 2};
    }
    // If some costs can be negative, call
    // this before maxflow:
    void setpi(int s) { // (otherwise,
        // leave this out)
        fill(all(pi), INF);
        pi[s] = 0;
        int it = N, ch = 1;
        ll v;
        while (ch-- && it--)
            rep(i, 0, N) if (pi[i] !=
                INF) for (edge& e
                    :
                        ed[i])
                            if (e.cap) if ((v = pi[i] + e.cost)
                                <
                                    pi[e.to]) pi[e.to] =
                                        v,
                                            ch = 1;
            assert(it >= 0); // negative cost
            cycle
        }
};
```

6.2 Dinic

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max |cap|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{VE})$ for bipartite matching.
Time: $O(VE \log U)$

```

const int64_t inf = 1LL << 60;
struct Dinic {
    struct Edge {
        int to, rev;
        int64_t flow, w;
        int id;
    };
    int n, s, t, mxid;
    vector<int> d, done;
    vector<int64_t> flow_through;
    vector<vector<Edge>> g;
    Dinic() {}
    Dinic(int _n) {
        n = _n + 10;
        mxid = 0;
        g.resize(n);
    }
    void add_edge(int u, int v, int64_t w
        , int id = -1) {
        Edge a = {v, (int)g[v].size(), 0,
            w, id};
        Edge b = {u, (int)g[u].size(), 0,
            0, -2}; // cap(b) = w for
            bidirectional
        g[u].emplace_back(a);
        g[v].emplace_back(b);
        if (id >= 0) mxid = max(mxid, id);
    }
    bool bfs() {
        d.assign(n, -1);
        d[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto &e : g[u]) {
                int v = e.to;
                if (d[v] == -1 && e.flow < e
                    .w) {
                    d[v] = d[u] + 1;
                    q.push(v);
                }
            }
        }
    }
```

```

    }
    return d[t] != -1;
}
int64_t dfs(int u, int64_t flow) {
    if (u == t) return flow;
    for (int &i = done[u]; i < (int)g[u].size(); i++) {
        Edge &e = g[u][i];
        if (e.w <= e.flow) continue;
        int v = e.to;
        if (d[v] == d[u] + 1) {
            int64_t nw = dfs(v, min(flow, e.w - e.flow));
            if (nw > 0) {
                e.flow += nw;
                g[v][e.rev].flow -= nw;
                return nw;
            }
        }
    }
    return 0;
}
int64_t max_flow(int _s, int _t) {
    s = _s;
    t = _t;
    int64_t flow = 0;
    while (bfs()) {
        done.assign(n, 0);
        while (int64_t nw = dfs(s, inf)) {
            flow += nw;
        }
        flow_through.assign(mxid+10, int64_t(0));
        for (int i = 0; i < n; i++) {
            for (auto &e : g[i]) {
                if (e.id >= 0) {
                    flow_through[e.id] = e.flow;
                }
            }
        }
        return flow;
    }
};

```

6.3 HopcroftKarp

Description: Fast bipartite matching. g is the adjacency list of the left partition. r is a vector of -1s of size equal to the right partition; after the call, $r[i]$ is the left-side match of right vertex i , or -1 if unmatched. Returns the size of the maximum matching.

Time: $O(E\sqrt{V})$

```

int hopcroftKarp(vector<vi>& g, vi& r) {
    int n = sz(g), res = 0;
    vi l(n, -1), q(n), d(n);
    auto dfs = [&](auto f, int u) -> bool {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u])
            if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        return 0;
    };
    for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
        rep(i, 0, n) if (l[i] == -1) q[t++] = i, d[i] = 1;
        rep(i, 0, t) for (int v : g[q[i]]) {
            if (r[v] == -1) f = 1;
            else if (!d[r[v]])
                d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
        }
        if (!f) return res;
        rep(i, 0, n) if (l[i] == -1) res += dfs(dfs, i);
    }
}
// --- Shortened usage ---
int main() {
    int n, m, k; cin >> n >> m >> k;
    vi r(n + m, -1); vector<vi> g(n + m);
}

```

```

rep(i, 0, k) {
    int u, v; cin >> u >> v;
    g[u-1].push_back(v + n - 1); // Map right side to [n, n+m]
}
int matchingSize = hopcroftKarp(g, r);
// r[i] stores the left-side match for right-side node i
for (int i = n; i < n + m; i++)
    if (r[i] != -1) cout << r[i] + 1 << " " << i - n + 1 << endl;
}

```

7 String

7.1 Trie

```

struct trienode {
    bool endmark;
    vector<int> child;
    trienode(int sz) {
        endmark = false;
        child.resize(sz, 0);
    }
};
struct trie {
    int sz;
    char fst;
    vector<trienode> nodes;
    trie(int alpha_sz, char alpha_start) {
        sz = alpha_sz;
        fst = alpha_start;
        // root at idx 0
        nodes.pb(trienode(sz));
    }
    void insert(string& s) {
        int cur = 0;
        for (char c : s) {
            if (!nodes[cur].child[c - fst]) {
                nodes[cur].child[c - fst] = nodes.size();
                nodes.pb(trienode(sz));
            }
            cur = nodes[cur].child[c - fst];
        }
        nodes[cur].endmark = true;
    }
    bool search(string& s) {
        int cur = 0;
        for (char c : s) {
            if (!nodes[cur].child[c - fst])
                return false;
            cur = nodes[cur].child[c - fst];
        }
        return nodes[cur].endmark;
    }
    bool erase(string& s) {
        int cur = 0;
        for (char c : s) {
            if (!nodes[cur].child[c - fst])
                return false;
            cur = nodes[cur].child[c - fst];
        }
        if (!nodes[cur].endmark) return false;
        nodes[cur].endmark = false;
        return true;
    }
};

```

7.2 Bit Trie

```

#define MAXN 100000011
int trie[MAXN][2], triesz[MAXN], nodes = 0, bitsz = 30;
void insert(int x) {
    int cur = 0;
    for (int i = bitsz; i >= 0; --i) {
        int child = x & (1 << i) ? 1 : 0;
        if (!trie[cur][child]) trie[cur][child] = ++nodes;
        triesz[cur]++;
        cur = trie[cur][child];
    }
    triesz[cur]++;
}
void erase(int x) {
    int cur = 0;
}

```

```

for (int i = bitsz; i >= 0; --i) {
    int child = x & (1 << i) ? 1 : 0;
    if (!triesz[cur] || !trie[cur][child]) return;
    triesz[cur]--;
    cur = trie[cur][child];
}
triesz[cur]--;
}
// max xor with x
int get_max(int x) {
    int cur = 0, ans = 0;
    for (int i = bitsz; i >= 0; --i) {
        int child = x & (1 << i) ? 1 : 0;
        child = 1 - child; // remove for get_min()
        if (!trie[cur][child] || !triesz[trie[cur][child]]) child = 1 - child;
        cur = trie[cur][child];
        ans |= child << i;
    }
    return ans ^ x;
}
void deleteall(int root) {
    if (trie[root][0]) deleteall(trie[root][0]);
    if (trie[root][1]) deleteall(trie[root][1]);
    trie[root][0] = trie[root][1] = 0;
    triesz[root] = 0;
}
void cleanup() {
    nodes = 0;
    deleteall(0);
}

```

7.3 Z Algorithm

```

VI Z(string& s) {
    int n = s.size();
    VI z(n);
    int l = 0, r = 0;
    for (int i = 0; i < n; ++i) {
        if (i <= r) z[i] = min(z[i - l], r - i + 1);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) l = i, r = i + z[i]++;
    }
    return z;
}

```

7.4 KMP

Description: $pi[x]$ computes the length of the longest prefix of s that ends at x , other than $s[0...x]$ itself (abacaba -> 0010123). Can be used to find all occurrences of a string. Time: $O(n)$

```

vi pi(const string& s) {
    vi p(sz(s));
    rep(i, 1, sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i, sz(p)-sz(s), sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}

```

7.5 Suffix Array

Description: Builds suffix array for a string. $sa[i]$ is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and $sa[0] = n$. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: $lcp[i] = lcp(sa[i], sa[i-1])$, $lcp[0] = 0$. The input string must not contain any nul chars. Time: $O(n \log n)$

```

void suffix_array(string s) {
    s += "$";
}

```

```

int32_t n=s.size(),N=max(n,int32_t
(260));
vector<int32_t> sa(n),R(n);
for(int32_t i=0;i<n;i++) sa[i]=i, R
[i]=s[i];
for(int32_t k=0;k<n;k?(k<=1):k++){
vector<int32_t> nR(n),cnt(N);
auto nsa=sa;
for(int32_t i=0;i<n;i++) nsa[i]=(
nsa[i]-k+n)%n, cnt[R[i]]++;
for(int32_t i=1;i<N;i++) cnt[i]+=
cnt[i-1];
for(int32_t i=n-1;i>=0;i--) sa[--
cnt[R[nsa[i]]]]=nsa[i];
for(int32_t i=1,r=0;i<n;i++){
r+=(R[sa[i]]!=R[sa[i-1]] || R[(
sa[i]+k)%n]!=R[(sa[i-1]+k)%n]);
nR[sa[i]]=r;
}
swap(R,nR);
}
vector<int32_t> lcp(n-1);
for(int32_t i=0,k=0;i<n-1;i++){
int32_t pi=R[i];
int32_t j=sa[pi-1];
while(i+k<n && j+k<n && s[i+k]==s[
j+k]) k++;
lcp[pi-1]=k;
if(k) k--;
}
}

```

7.6 Suffix Automaton

```

const int N = 3e5 + 9;
struct SuffixAutomaton {
    struct node {
        int len, link, firstpos;
        map<char, int> nxt;
    };
    int sz, last;
    vector<node> t;
    vector<int> terminal;
    vector<long long> dp;
    vector<vector<int>> g;
    SuffixAutomaton() {}
    SuffixAutomaton(int n) {
        t.resize(2 * n); terminal.resize
(2 * n, 0);
        dp.resize(2 * n, -1); sz = 1;
        last = 0;
        g.resize(2 * n);
        t[0].len = 0; t[0].link = -1; t
[0].firstpos = 0;
    }
    void extend(char c) {
        int p = last;
        if (t[p].nxt.count(c)) {
            int q = t[p].nxt[c];
            if (t[q].len == t[p].len +
1) {
                last = q;
                return;
            }
            int clone = sz++;
            t[clone] = t[q];
            t[clone].len = t[p].len + 1;
            t[q].link = clone;
            last = clone;
            while (p != -1 && t[p].nxt[c
] == q) {
                t[p].nxt[c] = clone;
                p = t[p].link;
            }
            return;
        }
        int cur = sz++;
        t[cur].len = t[last].len + 1;
        t[cur].firstpos = t[cur].len;
        p = last;
        while (p != -1 && !t[p].nxt.
count(c)) {
            t[p].nxt[c] = cur;
            p = t[p].link;
        }
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].nxt[c];

```

```

            if (t[p].len + 1 == t[q].len
) t[cur].link = q;
            else {
                int clone = sz++;
                t[clone] = t[q];
                t[clone].len = t[p].len
+ 1;
                while (p != -1 && t[p].
nxt[c] == q) {
                    t[p].nxt[c] = clone;
                    p = t[p].link;
                }
                t[q].link = t[cur].link
= clone;
            }
            last = cur;
        }
    }
    void build_tree() {
        for (int i = 1; i < sz; i++) g[t
[i].link].push_back(i);
    }
    void build(string &s) {
        for (auto x: s) {
            extend(x);
            terminal[last] = 1;
        }
        build_tree();
    }
    long long cnt(int i) {
        if (dp[i] != -1) return dp[i];
        long long ret = terminal[i];
        for (auto &x: g[i]) ret += cnt(x
);
        return dp[i] = ret;
    }
};

```

7.7 Manacher

Description: For each position in a string, computes $p[0][i]$ = half length of longest even palindrome around pos i , $p[1][i]$ = longest odd (half rounded down). Time: $O(N)$

```

array<vector<int>, 2> manacher(const
string& s) {
    int n = sz(s);
    array<vector<int>, 2> p = {vector<int>(
n+1), vector<int>(n)};
    rep(z, 0, 2) for (int i=0, l=0, r=0; i < n
; i++) {
        int t = r-i+!z;
        if (i<r) p[z][i] = min(t, p[z][l+t])
;
        int L = i-p[z][i], R = i+p[z][i]-!z;
        while (L>=1 && R+1<n && s[L-1] == s[
R+1])
            p[z][i]++, L--, R++;
        if (R>r) l=L, r=R;
    }
    return p;
}

```

7.8 Aho Corasick

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(-, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. Time: construction takes $O(26N)$, where N = sum of length of patterns. find(x) is $O(N)$, where N = length of x. findAll is $O(NM)$

```

struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; //
change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1,
end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof
(next)); }
    };
};

```

```

vector<Node> N;
vi backp;
void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
        int& m = N[n].next[c - first];
        if (m == -1) { n = m = sz(N); N.
emplace_back(-1); }
        else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
}
AhoCorasick(vector<string>& pat) : N
(1, -1) {
    rep(i, 0, sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);
    queue<int> q;
    for (q.push(0); !q.empty(); q.pop())
    {
        int n = q.front(), prev = N[n].
back;
        rep(i, 0, alpha) {
            int &ed = N[n].next[i], y = N[
prev].next[i];
            if (ed == -1) ed = y;
            else {
                N[ed].back = y;
                (N[ed].end == -1 ? N[ed].end :
backp[N[ed].start])
                = N[y].end;
                N[ed].nmatches += N[y].
nmatches;
                q.push(ed);
            }
        }
    }
}
vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}
vector<vi> findAll(vector<string>& pat
, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].
push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};

```

7.9 Min Lexic Rotation

```

int mnlex(string& s) {
    int n = s.size(), sz = 2 * n, l = 0, j
= 1, k = 0;
    while(l + k < sz && j + k < sz) {
        int a = ((l + k >= n) ? s[l + k - n]
: s[l + k]);
        int b = ((j + k >= n) ? s[j + k - n]
: s[j + k]);
        if (a == b) k++;
        else if (a > b) { // if maxlex then (
a<b)
            l = l + k + 1; k = 0;
            if (l <= j) { l = j + 1; }
        }
        else {
            j = j + k + 1;
            k = 0;
            if (j <= l) { j = l + 1; }
        }
    }
}

```



```

}
return min(l, j);
}

```

7.10 Longest Palindrome

```

void lp(string& s) {
    string t = s;
    reverse(t.begin(), t.end());
    int n = s.size();
    Hash sh(s), th(t);
    auto f = [&](int L, int R) {
        auto ss = sh.geth(L, R);
        auto tt = th.geth(n - R + 1, n - L + 1);
        return (ss == tt);
    };
    vector<int> R(n, 1), L(n, 1);
    for (int i = n - 1, r = n - 1; i >= 0; i--) {
        if (r < n - 1 && s[i] == s[r + 1]) {
            r++;
        } else {
            while (!f(i + 1, r + 1)) {
                r--;
            }
        }
        R[i] = max(R[i], r - i + 1);
    }
    for (int i = 0, l = 0; i < n; i++) {
        if (l > 0 && s[i] == s[l - 1]) {
            l--;
        } else {
            while (!f(l + 1, i + 1)) {
                l++;
            }
        }
        L[i] = max(L[i], i - l + 1);
    }
}

```

7.11 Wildcard Matching

```

//find occurrences of t in s where '?'s
//are automatically matched with any
//character
//res[i + m - 1] = sum_j=0 to m - 1 {s[i
//+ j] * t[j] * (s[i + j] - t[j])^2
vector<int> string_matching(string &s,
    string &t) {
    int n = s.size(), m = t.size();
    vector<int> s1(n), s2(n), s3(n);
    for (int i = 0; i < n; i++) s1[i] = s[i]
        == '?' ? 0 : s[i] - 'a' + 1; //
        assign any non zero number for non
        '?'s
    for (int i = 0; i < n; i++) s2[i] = s1[i]
        * s1[i];
    for (int i = 0; i < n; i++) s3[i] = s1[i]
        * s2[i];
    vector<int> t1(m), t2(m), t3(m);
    for (int i = 0; i < m; i++) t1[i] = t[i]
        == '?' ? 0 : t[i] - 'a' + 1;
    for (int i = 0; i < m; i++) t2[i] = t1[i]
        * t1[i];
    for (int i = 0; i < m; i++) t3[i] = t1[i]
        * t2[i];
    reverse(t1.begin(), t1.end());
    reverse(t2.begin(), t2.end());
    reverse(t3.begin(), t3.end());
    vector<int> s1t3 = multiply(s1, t3);
    vector<int> s2t2 = multiply(s2, t2);
    vector<int> s3t1 = multiply(s3, t1);
    vector<int> res(n);
    for (int i = 0; i < n; i++) res[i] =
        s1t3[i] - s2t2[i] * 2 + s3t1[i];
    vector<int> oc;
    for (int i = m - 1; i < n; i++) if (res[i]
        == 0) oc.push_back(i - m + 1);
    return oc;
}

```

7.12 Hashing

```

const int SZ = int(1e6) + 9;
const int B1 = 151, B2 = 239;
const int mod1 = 127657753, mod2 =
    987654319;
vector<int> P1(SZ, 1), P2(SZ, 1);
void pow_cal() {

```

```

    for (int i = 1; i < SZ; i++) {
        P1[i] = (P1[i - 1] * B1) % mod1;
        P2[i] = (P2[i - 1] * B2) % mod2;
    }
}
class Hash {
public:
    int n;
    vector<int> H1, H2;
    Hash(const string& s) {
        n = s.size();
        H1.assign(n + 1, 0);
        H2.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) {
            H1[i] = (H1[i - 1] + ((s[i - 1] -
                'a' + 1) * P1[i]) % mod1) % mod1;
            H2[i] = (H2[i - 1] + ((s[i - 1] -
                'a' + 1) * P2[i]) % mod2) % mod2;
        }
    }
    pair<int, int> geth(int l, int r) {
        int h1 = (((H1[r] - H1[l - 1] + mod1)
            ) % mod1) * P1[SZ - l] % mod1;
        int h2 = (((H2[r] - H2[l - 1] + mod2)
            ) % mod2) * P2[SZ - l] % mod2;
        return make_pair(h1, h2);
    }
    pair<int, int> geth() { return geth(1,
        n); }
};

```

7.13 2D Hash

```

struct Hashing {
    vector<vector<int>> hs;
    vector<int> PWX, PWY;
    int n, m;
    static const int PX = 3731, PY = 2999,
        mod = 998244353;
    Hashing() {}
    Hashing(vector<string>& s) {
        n = (int)s.size(), m = (int)s[0].
            size();
        hs.assign(n + 1, vector<int>(m + 1,
            0));
        PWX.assign(n + 1, 1);
        PWY.assign(m + 1, 1);
        for (int i = 0; i < n; i++) PWX[i +
            1] = 1LL * PWX[i] * PX % mod;
        for (int i = 0; i < m; i++) PWY[i +
            1] = 1LL * PWY[i] * PY % mod;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                hs[i + 1][j + 1] = s[i][j] - 'a'
                    + 1;
            }
        }
        for (int i = 0; i <= n; i++) {
            for (int j = 0; j < m; j++) {
                hs[i][j + 1] = (hs[i][j + 1] + 1
                    LL * hs[i][j] * PY % mod) % mod;
            }
        }
        for (int i = 0; i < n; i++) {
            for (int j = 0; j <= m; j++) {
                hs[i + 1][j] = (hs[i + 1][j] + 1
                    LL * hs[i][j] * PX % mod) % mod;
            }
        }
    }
    int get_hash(int x1, int y1, int x2,
        int y2) { // 1-indexed
        assert(1 <= x1 && x1 <= x2 && x2 <=
            n);
        assert(1 <= y1 && y1 <= y2 && y2 <=
            m);
        x1--;
        y1--;
        int dx = x2 - x1, dy = y2 - y1;
        return (1LL * (hs[x2][y2] - 1LL * hs
            [x2][y1] * PWY[dy] % mod + mod) %
            mod -
            1LL * (hs[x1][y2] - 1LL * hs
            [x1][y1] * PWY[dy] % mod + mod) %
            mod *
            PWX[dx] % mod +
            mod) %
            mod;
    }
};

```

```

}
int get_hash() { return get_hash(1, 1,
    n, m); }
};

```

8 Numerical

8.1 FFT

Description: `fft(a)` computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x - i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n , reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod. Time: $O(N \log N)$ with $N = |A| + |B|$ (1s for $N = 2^{22}$)

```

using C = complex<double>;
using vd = vector<double>;
void fft(vector<C>& a) {
    int n = a.size(), L = 31 -
        __builtin_clz(n);
    static vector<complex<long double>>
        R(2, 1);
    static vector<C> rt(2, 1);
    static int kBuilt = 2;
    for (; kBuilt < n; kBuilt *= 2) {
        R.resize(2 * kBuilt);
        rt.resize(2 * kBuilt);
        auto x = polar(1.0L, acos(-1.0L))
            / kBuilt;
        for (int i = kBuilt; i < 2 *
            kBuilt; i++)
            rt[i] = R[i] = (i & 1) ? R[i]
                / 2 : x * R[i / 2];
    }
    vector<int> rev(n);
    for (int i = 0; i < n; i++) rev[i] =
        (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; i++) if (i <
        rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 *
            k)
            for (int j = 0; j < k; j++)
            {
                // Hand-rolled complex
                multiply (~25% faster than rt[j+k]*a
                [i+j+k])
                auto x = (double*)&rt[j
                    + k];
                auto y = (double*)&a[i +
                    j + k];
                C z(x[0] * y[0] - x[1] *
                    y[1],
                    x[0] * y[1] + x[1] *
                    y[0]);
                a[i + j + k] = a[i + j]
                    - z;
                a[i + j] += z;
            }
    }
    vector<ll> conv(const vector<ll>& a,
        const vector<ll>& b) {
        if (a.empty() || b.empty()) return
            {};
        vector<ll> res(a.size() + b.size() -
            1);
        int L = 32 - __builtin_clz((int)res.
            size()), n = 1 << L;
        vector<C> in(n), out(n);
        copy(a.begin(), a.end(), in.begin())
            ;
        for (int i = 0; i < (int)b.size(); i
            ++)) in[i].imag(b[i]);
        fft(in);
        for (C& x : in) x *= x;
        for (int i = 0; i < n; i++) out[i] =
            in[-i & (n - 1)] - conj(in[i]);
        fft(out);
        for (int i = 0; i < (int)res.size();
            i++) {
            res[i] = (ll)round(imag(out[i])
                / (4 * n));
        }
    }
}

```

```

}
return res;
}

```

8.2 FFT With mod M

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $O(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

```

template <ll M>
vector<ll> convMod(vector<ll>& a, vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    vector<ll> res(a.size() + b.size() - 1);
    int B = 32 - __builtin_clz((int)res.size());
    int cut = (int)sqrt((double)M);
    vector<C> fa(n), fb(n), outs(n), outl(n);
    for (int i = 0; i < (int)a.size(); i++) fa[i] = C((int)a[i] / cut, (int)a[i] % cut);
    for (int i = 0; i < (int)b.size(); i++) fb[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(fa); fft(fb);
    for (int i = 0; i < n; i++) {
        int j = -i & (n - 1);
        outl[j] = (fa[i] + conj(fa[j])) * fb[j] / (2.0 * n);
        outs[j] = (fa[i] - conj(fa[j])) * fb[j] / (2.0 * n) / C(0, 1);
    }
    fft(outl); fft(outs);
    for (int i = 0; i < (int)res.size(); i++) {
        ll av = llround(real(outl[i]));
        ll bv = llround(imag(outl[i])) + llround(real(outs[i]));
        ll cv = llround(imag(outs[i]));
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}

```

8.3 NTT

Description: $\text{ntt}(a)$ computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes. For manual convolution: NTT the inputs, multiply pointwise, divide by n , reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.
Time: $O(N \log N)$

```

const long long MOD = 998244353, root = 3;
void ntt(long long* x, long long* temp, long long* roots, int N, int skip) {
    if (N == 1) return;
    int n2 = N / 2;
    ntt(x, temp, roots, n2, skip * 2);
    ntt(x + skip, temp, roots, n2, skip * 2);
    for (int i = 0; i < N; i++) temp[i] = x[i * skip];
    for (int i = 0; i < n2; i++) {
        long long s = temp[2 * i], t = temp[2 * i + 1] * roots[skip * i];
        x[skip * i] = (s + t) % MOD;
        x[skip * (i + n2)] = ((s - t) % MOD + MOD) % MOD;
    }
}
void ntt(vector<long long>& x, bool inv = false) {
    long long e = modpow(root, (MOD - 1) / x.size(), MOD);
    if (inv) e = modpow(e, MOD - 2, MOD);
    vector<long long> roots(x.size(), 1), temp = roots;

```

```

    for (size_t i = 1; i < x.size(); i++) roots[i] = roots[i - 1] * e % MOD;
    ntt(&x[0], &temp[0], &roots[0], x.size(), 1);
}
vector<long long> conv(vector<long long> a, vector<long long> b) {
    int s = a.size() + b.size() - 1;
    if (s <= 0) return {};
    int L = s > 1 ? 32 - __builtin_clz(s - 1) : 0, n = 1 << L;
    if (s <= 200) {
        vector<long long> c(s);
        for (size_t i = 0; i < a.size(); i++) {
            for (size_t j = 0; j < b.size(); j++) {
                c[i + j] = (c[i + j] + a[i] * b[j]) % MOD;
            }
        }
        return c;
    }
    a.resize(n); ntt(a);
    b.resize(n); ntt(b);
    vector<long long> c(n);
    long long d = modpow(n, MOD - 2, MOD);
    for (int i = 0; i < n; i++) c[i] = a[i] * b[i] % MOD * d % MOD;
    ntt(c, true);
    c.resize(s);
    return c;
}
vector<long long> mul(const vector<long long>& a, const vector<long long>& b, int n) {
    vector<long long> c = conv(a, b);
    if ((int)c.size() > n + 1) c.resize(n + 1);
    return c;
}
vector<long long> power(vector<long long> > f, long long m, int n) {
    vector<long long> r = {1};
    if ((int)f.size() > n + 1) f.resize(n + 1);
    for (; m > 0; m >= 1, f = mul(f, f, n)) {
        if (m & 1) r = mul(r, f, n);
        return r;
    }
}

```

8.4 FWHT

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{x \oplus y = z} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two. Time: $O(N \log N)$

```

void FST(vector<int>& a, bool inv) {
    for (int n = a.size(), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step)
            for (int j = i; j < i + step; j++) {
                int &u = a[j], &v = a[j + step];
                tie(u, v) = inv ? pair<int, int>(v - u, u) : pair<int, int>(v, u + v); // AND
                // inv ? pair<int, int>(v, u - v) : pair<int, int>(u + v, u); // OR
                // OR // include-line
                // pair<int, int>(u + v, u - v); // XOR
                // include-line
            }
        // if (inv) for (int& x : a) x /= a.size(); // XOR only // include-line
    }
}
vector<int> conv(vector<int> a, vector<int> b) {
    FST(a, 0); FST(b, 0);
    for (int i = 0; i < a.size(); i++) a[i] *= b[i];
    FST(a, 1); return a;
}

```

```

}

```

9 Linear Algebra

9.1 The Matrix Class

```

struct Mat {
    int n, m;
    vector<vector<int>> a;
    Mat() {}
    Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0));}
    Mat(vector<vector<int>> > v) {n = v.size(); m = n ? v[0].size() : 0; a = v;}
    inline void make_unit() {
        assert(n == m);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) a[i][j] = i == j;
        }
    }
    inline Mat operator + (const Mat &b) {
        assert(n == b.n && m == b.m);
        Mat ans = Mat(n, m);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
            }
        }
        return ans;
    }
    inline Mat operator - (const Mat &b) {
        assert(n == b.n && m == b.m);
        Mat ans = Mat(n, m);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
            }
        }
        return ans;
    }
    inline Mat operator * (const Mat &b) {
        assert(m == b.n);
        Mat ans = Mat(n, b.m);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < b.m; j++) {
                for (int k = 0; k < m; k++) {
                    ans.a[i][j] = (ans.a[i][j] + LL * a[i][k] * b.a[k][j] % mod) % mod;
                }
            }
        }
        return ans;
    }
    inline Mat pow(long long k) {
        assert(n == m);
        Mat ans(n, n), t = a; ans.make_unit();
        while (k) {
            if (k & 1) ans = ans * t;
            t = t * t;
            k >>= 1;
        }
        return ans;
    }
    inline Mat& operator += (const Mat& b) {
        return *this = (*this) + b;
    }
    inline Mat& operator -= (const Mat& b) {
        return *this = (*this) - b;
    }
    inline Mat& operator *= (const Mat& b) {
        return *this = (*this) * b;
    }
    inline bool operator == (const Mat& b) {
        return a == b.a;
    }
    inline bool operator != (const Mat& b) {
        return a != b.a;
    }
};

```

9.2 Gaussian Elimination

Description: Input : Mat a of size n x (m+1), where the last column is the RHS (b vector) i.e. a represents the augmented matrix [A — b] Output: Fills 'ans' with solution vector x such that $A \cdot x = b \pmod{p}$ Returns -1 $\rightarrow$ no solution exists Returns 0 $\rightarrow$ unique solution Returns k $\rightarrow$ infinitely many solutions with k free variables

Usage:

Mat aug(n, m + 1); fill aug.a[i][j] with coefficients, aug.a[i][m] with RHS vector<int> sol; int result = Gauss(aug, sol); if (result == -1) // no solution else if (result == 0) // unique solution in sol else // result = number of free variables

```
int Gauss(Mat a, vector<int> &ans) {
    int n = a.n, m = a.m - 1;
    vector<int> pos(m, -1);
    int free_var = 0;
    const long long MODSQ = (long long)mod
        * mod;
    int rank = 0;
    for (int col = 0, row = 0; col < m &&
        row < n; col++) {
        int mx = row;
        for (int k = row; k < n; k++) if (a.
            a[k][col] > a.a[mx][col]) mx = k;
        if (a.a[mx][col] == 0) continue;
        for (int j = col; j <= m; j++) swap(
            a.a[mx][j], a.a[row][j]);
        pos[col] = row;
        int inv = power(a.a[row][col], mod -
            2);
        for (int i = 0; i < n && inv; i++) {
            if (i != row && a.a[i][col]) {
                int x = (1LL * a.a[i][col] * inv
                    ) % mod;
                for (int j = col; j <= m && x; j
                    ++){
                    if (a.a[row][j]) a.a[i][j] = (
                        MODSQ + a.a[i][j] - 1LL * a.a[row][
                            j] * x) % mod;
                }
            }
        }
        row++; ++rank;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++) {
        if (pos[i] == -1) free_var++;
        else ans[i] = (1LL * a.a[pos[i]][m]
            * power(a.a[pos[i]][i], mod - 2)) %
            mod;
    }
    for (int i = 0; i < n; i++) {
        long long val = 0;
        for (int j = 0; j < m; j++) val = (
            val + 1LL * ans[j] * a.a[i][j]) %
            mod;
        if (val != a.a[i][m]) return -1;
    }
    return free_var;
}
```

9.3 Determinant

Description: Determinant of a matrix.

Usage:

Mat a(n, n); // fill a.a[i][j] int d = Det(a); // d = det(A) mod p

```
int Det(Mat a) {
    assert(a.n == a.m);
    int n = a.n;
    const long long MODSQ = (long long)mod
        * mod;
    int det = 1;
    for (int col = 0, row = 0; col < n &&
        row < n; col++) {
        int mx = row;
        for (int k = row; k < n; k++) if (a.
            a[k][col] > a.a[mx][col]) mx = k;
        if (a.a[mx][col] == 0) { det = 0;
            continue; }
        for (int j = col; j < n; j++) swap(a.
            a[mx][j], a.a[row][j]);
        if (row != mx) det = (det == 0 ? 0 :
            mod - det);
```

```
det = 1LL * det * a.a[row][col] %
    mod;
    int inv = power(a.a[row][col], mod -
        2);
    for (int i = 0; i < n && inv; i++) {
        if (i != row && a.a[i][col]) {
            int x = (1LL * a.a[i][col] * inv
                ) % mod;
            for (int j = col; j < n && x; j
                ++){
                if (a.a[row][j]) a.a[i][j] = (
                    MODSQ + a.a[i][j] - 1LL * a.a[row][
                        j] * x) % mod;
            }
        }
    }
    row++;
}
return det;
}
```

9.4 Matrix Inverse

Description: Inverse of a matrix

Usage:

Mat a(n, n), inv; // fill a.a[i][j] if (Inv(a, inv)) { /* use inv */ } else { /* singular */ }

```
bool Inv(Mat a, Mat &ans) {
    assert(a.n == a.m);
    int n = a.n;
    vector<int> col(n);
    Mat tmp(n, n); tmp.make_unit();
    for (int i = 0; i < n; i++) col[i] = i
        ;
    for (int i = 0; i < n; i++) {
        int r = i, c = i;
        for (int j = i; j < n; j++)
            for (int k = i; k < n; k++)
                if (a.a[j][k]) { r = j; c = k;
                    goto found; }
        return false;
    found:
        a.a[i].swap(a.a[r]); tmp.a[i].swap(
            tmp.a[r]);
        for (int j = 0; j < n; j++) { swap(a.
            a[j][i], a.a[j][c]); swap(tmp.a[j
                ][i], tmp.a[j][c]); }
        swap(col[i], col[c]);
        int v = power(a.a[i][i], mod - 2);
        for (int j = i + 1; j < n; j++) {
            int f = 1LL * a.a[j][i] * v % mod;
            a.a[j][i] = 0;
            for (int k = i + 1; k < n; k++) {
                a.a[j][k] -= 1LL * f * a.a[i][k]
                    % mod;
                if (a.a[j][k] < 0) a.a[j][k] +=
                    mod;
            }
            for (int k = 0; k < n; k++) {
                tmp.a[j][k] -= 1LL * f * tmp.a[i
                    ][k] % mod;
                if (tmp.a[j][k] < 0) tmp.a[j][k]
                    += mod;
            }
        }
        for (int j = i + 1; j < n; j++) a.a[
            i][j] = 1LL * a.a[i][j] * v % mod;
        for (int j = 0; j < n; j++) tmp.a[i
            ][j] = 1LL * tmp.a[i][j] * v % mod;
        a.a[i][i] = 1;
    }
    for (int i = n - 1; i > 0; --i) {
        for (int j = 0; j < i; j++) {
            int v = a.a[j][i];
            for (int k = 0; k < n; k++) {
                tmp.a[j][k] -= 1LL * v * tmp.a[i
                    ][k] % mod;
                if (tmp.a[j][k] < 0) tmp.a[j][k]
                    += mod;
            }
        }
    }
    ans = Mat(n, n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            ans.a[col[i]][col[j]] = tmp.a[i][j
                ];
}
```

```
return true;
}
```

9.5 Xor Basis

```
const int LOG_K = 60; // Adjust as
needed
struct XorBasis {
    vector<int64_t> basis;
    int64_t N = 0, tmp = 0;
    void add(int64_t x) {
        N++;
        tmp |= x;
        for (auto& i : basis) {
            x = min(x, x ^ i);
        }
        if (!x) return;
        for (auto& i : basis) {
            if ((i ^ x) < i) {
                i ^= x;
            }
        }
        basis.push_back(x);
        sort(basis.begin(), basis.end());
    }
    int64_t size() { return (int64_t)basis.
        .size(); }
    void clear() {
        N = 0;
        tmp = 0;
        basis.clear();
    }
    bool possible(int64_t x) {
        for (auto& i : basis) {
            x = min(x, x ^ i);
        }
        return !x;
    }
    int64_t maxxor(int64_t x = 0) {
        for (auto& i : basis) {
            x = max(x, x ^ i);
        }
        return x;
    }
    int64_t minxor(int64_t x = 0) {
        for (auto& i : basis) {
            x = min(x, x ^ i);
        }
        return x;
    }
    int64_t cntxor(int64_t x) {
        if (!possible(x)) return 0;
        return (1LL << (N - size()));
    }
    int64_t sumOfAll() {
        int64_t ans = tmp * (1LL << (N - 1))
            ;
        return ans;
    }
    int64_t kth(int64_t k) {
        int64_t sz = size();
        if (k > (1LL << sz)) return -1;
        k--;
        int64_t ans = 0;
        for (int64_t i = 0; i < sz; i++) {
            if (k >> i & 1) {
                ans ^= basis[i];
            }
        }
        return ans;
    }
};
```

10 Geometry

10.1 Full Template

```
const int N = 3e5 + 9;
const double inf = 1e100;
const double PI = acos((double)-1.0);
const double eps = 1e-9;
// Returns -1 for negative, 0 for zero,
1 for positive (with epsilon
tolerance)
int sign(double x) { return (x > eps) -
    (x < -eps); }
struct PT {
    double x, y;
```

```

PT() { x = 0, y = 0; }
PT(double x, double y) : x(x), y(y) {}
PT(const PT &p) : x(p.x), y(p.y) {}
PT operator + (const PT &a) const {
    return PT(x + a.x, y + a.y); }
PT operator - (const PT &a) const {
    return PT(x - a.x, y - a.y); }
PT operator * (const double a) const {
    return PT(x * a, y * a); }
friend PT operator * (const double &a,
    const PT &b) { return PT(a * b.x,
    a * b.y); }
PT operator / (const double a) const {
    return PT(x / a, y / a); }
bool operator == (PT a) const { return
    sign(a.x - x) == 0 && sign(a.y - y)
    == 0; }
bool operator != (PT a) const { return
    !(*this == a); }
bool operator < (PT a) const { return
    sign(a.x - x) == 0 ? y < a.y : x <
    a.x; }
bool operator > (PT a) const { return
    sign(a.x - x) == 0 ? y > a.y : x >
    a.x; }
double norm() { return sqrt(x * x + y
    * y); }
double norm2() { return x * x + y * y;
    }
PT perp() { return PT(-y, x); }
double arg() { return atan2(y, x); }
// Returns a vector with norm r and
// having same direction
PT truncate(double r) {
    double k = norm();
    if (!sign(k)) return *this;
    r /= k;
    return PT(x * r, y * r);
}
};
istream &operator >> (istream &in, PT &p)
    { return in >> p.x >> p.y; }
ostream &operator << (ostream &out, PT &
    p) { return out << "(" << p.x << ",
    " << p.y << ")"; }
inline double dot(PT a, PT b) { return a
    .x * b.x + a.y * b.y; }
inline double dist2(PT a, PT b) { return
    dot(a - b, a - b); }
inline double dist(PT a, PT b) { return
    sqrt(dot(a - b, a - b)); }
inline double cross(PT a, PT b) { return
    a.x * b.y - a.y * b.x; }
inline double cross2(PT a, PT b, PT c) {
    return cross(b - a, c - a); }
inline int orientation(PT a, PT b, PT c)
    { return sign(cross(b - a, c - a))
    ; }
PT perp(PT a) { return PT(-a.y, a.x); }
PT rotateccw90(PT a) { return PT(-a.y, a
    .x); }
PT rotatecw90(PT a) { return PT(a.y, -a.
    x); }
PT rotateccw(PT a, double t) { return PT
    (a.x * cos(t) - a.y * sin(t), a.x *
    sin(t) + a.y * cos(t)); }
PT rotatecw(PT a, double t) { return PT(
    a.x * cos(t) + a.y * sin(t), -a.x *
    sin(t) + a.y * cos(t)); }
double SQ(double x) { return x * x; }
double rad_to_deg(double r) { return (r
    * 180.0 / PI); }
double deg_to_rad(double d) { return (d
    * PI / 180.0); }
// Returns angle between vectors a and b
// in radians
double get_angle(PT a, PT b) {
    double costheta = dot(a, b) / a.norm()
    / b.norm();
    return acos(max((double)-1.0, min((
    double)1.0, costheta)));
}
struct line {
    PT a, b; // goes through points a and
    b
    PT v; double c; // line form:
    direction vec [cross] (x, y) = c
    line() {}

```

```

// Direction vector v and offset c
line(PT v, double c) : v(v), c(c) {
    auto p = get_points();
    a = p.first; b = p.second;
}
// Equation ax + by + c = 0
line(double _a, double _b, double _c)
    : v({_b, -_a}), c(-_c) {
    auto p = get_points();
    a = p.first; b = p.second;
}
// Goes through points p and q
line(PT p, PT q) : v(q - p), c(cross(v
    , p)), a(p), b(q) {}
// Extract any two points from this
// line
pair<PT, PT> get_points() {
    PT p, q;
    double a_coef = -v.y, b_coef = v.x;
    // ax + by = c
    if (sign(a_coef) == 0) {
        p = PT(0, c / b_coef);
        q = PT(1, c / b_coef);
    } else if (sign(b_coef) == 0) {
        p = PT(c / a_coef, 0);
        q = PT(c / a_coef, 1);
    } else {
        p = PT(0, c / b_coef);
        q = PT(1, (c - a_coef) / b_coef);
    }
    return {p, q};
}
// Returns coefficients for ax + by +
// c = 0
array<double, 3> get_abc() {
    double a_coef = -v.y, b_coef = v.x;
    return {a_coef, b_coef, -c};
}
// 1 if on the left, -1 if on the
// right, 0 if on the line
int side(PT p) { return sign(cross(v,
    p) - c); }
// Line that is perpendicular to this
// and goes through point p
line perpendicular_through(PT p) {
    return {p, p + perp(v)}; }
// Translate the line by vector t
line translate(PT t) { return {v, c +
    cross(v, t)}; }
// Compare two points by orthogonal
// projection on this line
bool cmp_by_projection(PT p, PT q) {
    return dot(v, p) < dot(v, q); }
line shift_left(double d) {
    PT z = v.perp().truncate(d);
    return line(a + z, b + z);
}
};
// Minimum distance from point c to line
// through a and b
double dist_from_point_to_line(PT a, PT
    b, PT c) {
    return fabs(cross(b - a, c - a) / (b -
    a).norm());
}
// Returns true if point p is on line
// segment ab
bool is_point_on_seg(PT a, PT b, PT p) {
    if (fabs(cross(p - b, a - b)) < eps) {
        if (p.x < min(a.x, b.x) - eps || p.x
            > max(a.x, b.x) + eps) return
            false;
        if (p.y < min(a.y, b.y) - eps || p.y
            > max(a.y, b.y) + eps) return
            false;
        return true;
    }
    return false;
}
// Minimum distance point from point c
// to segment ab that lies on segment
// ab
PT project_from_point_to_seg(PT a, PT b,
    PT c) {
    double r = dist2(a, b);
    if (sign(r) == 0) return a;
    r = dot(c - a, b - a) / r;
    if (r < 0) return a;

```

```

    if (r > 1) return b;
    return a + (b - a) * r;
}
// Minimum distance from point c to
// segment ab
double dist_from_point_to_seg(PT a, PT b
    , PT c) {
    return dist(c,
    project_from_point_to_seg(a, b, c))
    ;
}
// 0 if not parallel, 1 if parallel, 2
// if collinear
int is_parallel(PT a, PT b, PT c, PT d)
    {
    double k = fabs(cross(b - a, d - c));
    if (k < eps) {
        if (fabs(cross(a - b, a - c)) < eps
            && fabs(cross(c - d, c - a)) < eps)
            return 2;
        else return 1;
    } else return 0;
}
// 1 if point is ccw to the line, 2 if
// point is cw to the line, 3 if point
// is on the line
int point_line_relation(PT a, PT b, PT p
    ) {
    int c = sign(cross(p - a, b - a));
    if (c < 0) return 1;
    if (c > 0) return 2;
    return 3;
}
// Intersection point between line ab
// and line cd assuming unique
// intersection exists
bool line_line_intersection(PT a, PT b,
    PT c, PT d, PT &ans) {
    double a1 = a.y - b.y, b1 = b.x - a.x,
        c1 = cross(a, b);
    double a2 = c.y - d.y, b2 = d.x - c.x,
        c2 = cross(c, d);
    double det = a1 * b2 - a2 * b1;
    if (det == 0) return 0;
    ans = PT((b1 * c2 - b2 * c1) / det, (
        c1 * a2 - a1 * c2) / det);
    return 1;
}
// Intersection point between segment ab
// and segment cd assuming unique
// intersection exists
bool seg_seg_intersection(PT a, PT b, PT
    c, PT d, PT &ans) {
    double oa = cross2(c, d, a), ob =
        cross2(c, d, b);
    double oc = cross2(a, b, c), od =
        cross2(a, b, d);
    if (oa * ob < 0 && oc * od < 0) {
        ans = (a * ob - b * oa) / (ob - oa);
        return 1;
    } else return 0;
}
// Custom change: Boolean intersection
// check (handles collinear overlap)
bool seg_seg_intersection(PT a, PT b, PT
    c, PT d) {
    double oa = cross2(c, d, a), ob =
        cross2(c, d, b);
    double oc = cross2(a, b, c), od =
        cross2(a, b, d);
    if (oa * ob < 0 && oc * od < 0) return
        true;
    // Checking orientation for very
    // precise calculation
    if (oa == 0 && is_point_on_seg(c, d, a
        )) return true;
    if (ob == 0 && is_point_on_seg(c, d, b
        )) return true;
    if (oc == 0 && is_point_on_seg(a, b, c
        )) return true;
    if (od == 0 && is_point_on_seg(a, b, d
        )) return true;
    return false;
}
// 0 if do not intersect, 1 if proper
// intersect, 2 if segment intersect
int seg_line_relation(PT a, PT b, PT c,
    PT d) {

```



```

double p = cross2(c, d, a);
double q = cross2(c, d, b);
if (sign(p) == 0 && sign(q) == 0)
    return 2;
else if (p * q < 0) return 1;
else return 0;
}
// Intersection between segment ab and
// line cd assuming unique intersection
// exists
bool seg_line_intersection(PT a, PT b,
    PT c, PT d, PT &ans) {
    bool k = seg_line_relation(a, b, c, d);
    ;
    assert(k != 2);
    if (k) line_line_intersection(a, b, c,
        d, ans);
    return k;
}
// Minimum distance from segment ab to
// segment cd
double dist_from_seg_to_seg(PT a, PT b,
    PT c, PT d) {
    PT dummy;
    if (seg_seg_intersection(a, b, c, d,
        dummy)) return 0.0;
    else return min({
        dist_from_point_to_seg(a, b, c),
        dist_from_point_to_seg(a, b, d),

        dist_from_point_to_seg(c, d, a),
        dist_from_point_to_seg(c, d, b)});
}
struct circle {
    PT p; double r;
    circle() {}
    circle(PT _p, double _r): p(_p), r(_r)
    {}
    circle(double x, double y, double _r):
        p(PT(x, y)), r(_r) {} // center (x
        , y) and radius r
    // Circumcircle of a triangle (three
    // points must be unique)
    circle(PT a, PT b, PT c) {
        b = (a + b) * 0.5;
        c = (a + c) * 0.5;
        line_line_intersection(b, b +
            rotatecw90(a - b), c, c +
            rotatecw90(a - c), p);
        r = dist(a, p);
    }
    // Inscribed circle of a triangle (
    // pass a bool just to differentiate
    // from circumcircle)
    circle(PT a, PT b, PT c, bool t) {
        line u, v;
        double m = atan2(b.y - a.y, b.x - a.
            x), n = atan2(c.y - a.y, c.x - a.x)
            ;
        u.a = a;
        u.b = u.a + (PT(cos((n + m)/2.0),
            sin((n + m)/2.0)));
        v.a = b;
        m = atan2(a.y - b.y, a.x - b.x), n =
            atan2(c.y - b.y, c.x - b.x);
        v.b = v.a + (PT(cos((n + m)/2.0),
            sin((n + m)/2.0)));
        line_line_intersection(u.a, u.b, v.a
            , v.b, p);
        r = dist_from_point_to_seg(a, b, p);
    }
    bool operator == (circle v) { return p
        == v.p && sign(r - v.r) == 0; }
    double area() { return PI * r * r; }
    double circumference() { return 2.0 *
        PI * r; }
};
// 0 if outside, 1 if on circumference,
// 2 if inside circle
int circle_point_relation(PT p, double r
    , PT b) {
    double d = dist(p, b);
    if (sign(d - r) < 0) return 2;
    if (sign(d - r) == 0) return 1;
    return 0;
}
// 0 if outside, 1 if on circumference,
// 2 if inside circle

```

```

int circle_line_relation(PT p, double r,
    PT a, PT b) {
    double d = dist_from_point_to_line(a,
        b, p);
    if (sign(d - r) < 0) return 2;
    if (sign(d - r) == 0) return 1;
    return 0;
}
// Compute intersection of line through
// a and b with circle centered at c
// with radius r > 0
vector<PT> circle_line_intersection(PT c
    , double r, PT a, PT b) {
    vector<PT> ret;
    b = b - a; a = a - c;
    double A = dot(b, b), B = dot(a, b);
    double C = dot(a, a) - r * r, D = B *
        B - A * C;
    if (D < -eps) return ret;
    ret.push_back(c + a + b * (-B + sqrt(D
        + eps)) / A);
    if (D > eps) ret.push_back(c + a + b *
        (-B - sqrt(D)) / A);
    return ret;
}
// 5 - outside and do not intersect, 4 -
// intersect outside in one point
// 3 - intersect in 2 points, 2 -
// intersect inside in one point, 1 -
// inside and do not intersect
int circle_circle_relation(PT a, double
    r, PT b, double R) {
    double d = dist(a, b);
    if (sign(d - r - R) > 0) return 5;
    if (sign(d - r - R) == 0) return 4;
    double l = fabs(r - R);
    if (sign(d - r - R) < 0 && sign(d - l)
        > 0) return 3;
    if (sign(d - l) == 0) return 2;
    if (sign(d - l) < 0) return 1;
    assert(0); return -1;
}
vector<PT> circle_circle_intersection(PT
    a, double r, PT b, double R) {
    if (a == b && sign(r - R) == 0) return
        {PT(1e18, 1e18)};
    vector<PT> ret;
    double d = sqrt(dist2(a, b));
    if (d > r + R || d + min(r, R) < max(r
        , R)) return ret;
    double x = (d * d - R * R + r * r) /
        (2 * d);
    double y = sqrt(r * r - x * x);
    PT v = (b - a) / d;
    ret.push_back(a + v * x + rotateccw90(
        v) * y);
    if (y > 0) ret.push_back(a + v * x -
        rotateccw90(v) * y);
    return ret;
}
// Returns two circle c1, c2 through
// points a, b and of radius r
// 0 if there is no such circle, 1 if
// one circle, 2 if two circle
int get_circle(PT a, PT b, double r,
    circle &c1, circle &c2) {
    vector<PT> v =
        circle_circle_intersection(a, r, b,
            r);
    int t = v.size();
    if (!t) return 0;
    c1.p = v[0], c1.r = r;
    if (t == 2) c2.p = v[1], c2.r = r;
    return t;
}
// Returns area of intersection between
// two circles
double circle_circle_area(PT a, double
    r1, PT b, double r2) {
    double d = (a - b).norm();
    if (r1 + r2 < d + eps) return 0;
    if (r1 + d < r2 + eps) return PI * r1
        * r1;
    if (r2 + d < r1 + eps) return PI * r2
        * r2;
    double theta_1 = acos((r1 * r1 + d * d
        - r2 * r2) / (2 * r1 * d));
    double theta_2 = acos((r2 * r2 + d * d

```

```

        - r1 * r1) / (2 * r2 * d));
    return r1 * r1 * (theta_1 - sin(2 *
        theta_1) / 2.0) + r2 * r2 * (
        theta_2 - sin(2 * theta_2) / 2.0);
}
double area_of_triangle(PT a, PT b, PT c
    ) {
    return fabs(cross(b - a, c - a) * 0.5)
        ;
}
// -1 if strictly inside, 0 if on the
// polygon, 1 if strictly outside
int is_point_in_triangle(PT a, PT b, PT
    c, PT p) {
    if (sign(cross(b - a, c - a)) < 0)
        swap(b, c);
    int c1 = sign(cross(b - a, p - a));
    int c2 = sign(cross(c - b, p - b));
    int c3 = sign(cross(a - c, p - c));
    if (c1 < 0 || c2 < 0 || c3 < 0) return
        1;
    if (c1 + c2 + c3 != 3) return 0;
    return -1;
}
double area(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans +=
        cross(p[i], p[(i + 1) % n]);
    return fabs(ans) * 0.5;
}
// 0 if cw, 1 if ccw
bool get_direction(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans +=
        cross(p[i], p[(i + 1) % n]);
    if (sign(ans) > 0) return 1;
    return 0;
}
// Returns the convex hull of the set of
// points p (Monotone Chain)
vector<PT> convex_hull(vector<PT> &p) {
    if (p.size() <= 1) return p;
    vector<PT> v = p;
    sort(v.begin(), v.end());
    vector<PT> up, dn;
    for (auto& pt : v) {
        while (up.size() > 1 && orientation(
            up[up.size() - 2], up.back(), pt)
            >= 0) up.pop_back();
        while (dn.size() > 1 && orientation(
            dn[dn.size() - 2], dn.back(), pt)
            <= 0) dn.pop_back();
        up.push_back(pt);
        dn.push_back(pt);
    }
    v = dn;
    if (v.size() > 1) v.pop_back();
    reverse(up.begin(), up.end());
    up.pop_back();
    for (auto& pt : up) v.push_back(pt);
    if (v.size() == 2 && v[0] == v[1]) v.
        pop_back();
    return v;
}
// Checks if convex or not
bool is_convex(vector<PT> &p) {
    bool s[3] = {0, 0, 0};
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        int k = (j + 1) % n;
        s[sign(cross(p[j] - p[i], p[k] - p[i]
            )) + 1] = 1;
        if (s[0] && s[2]) return 0;
    }
    return 1;
}
// -1 if strictly inside, 0 if on the
// polygon, 1 if strictly outside (O(
// log n))
int is_point_in_convex(vector<PT> &p,
    const PT& x) {
    int n = p.size(); assert(n >= 3);
    int a = orientation(p[0], p[1], x), b
        = orientation(p[0], p[n - 1], x);
    if (a < 0 || b > 0) return 1;
    int l = 1, r = n - 1;
    while (l + 1 < r) {

```

```

    int mid = (l + r) >> 1;
    if (orientation(p[0], p[mid], x) >= 0) l = mid;
    else r = mid;
}
int k = orientation(p[l], p[r], x);
if (k <= 0) return -k;
if (l == 1 && a == 0) return 0;
if (r == n - 1 && b == 0) return 0;
return -1;
}
bool is_point_on_polygon(vector<PT> &p,
    const PT& z) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        if (is_point_on_seg(p[i], p[(i + 1) % n], z)) return 1;
    }
    return 0;
}
// Returns 1e9 if the point is on the
// polygon (O(n))
int winding_number(vector<PT> &p, const
    PT& z) {
    if (is_point_on_polygon(p, z)) return
        1e9;
    int n = p.size(), ans = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        bool below = p[i].y < z.y;
        if (below != (p[j].y < z.y)) {
            auto orient = orientation(z, p[j],
                p[i]);
            if (orient == 0) return 0;
            if (below == (orient > 0)) ans +=
                below ? 1 : -1;
        }
    }
    return ans;
}
// -1 if strictly inside, 0 if on the
// polygon, 1 if strictly outside
int is_point_in_polygon(vector<PT> &p,
    const PT& z) {
    int k = winding_number(p, z);
    return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}
// id of the vertex having maximum dot
// product with z (O(log n))
int extreme_vertex(vector<PT> &p, const
    PT &z, const int top) {
    int n = p.size();
    if (n == 1) return 0;
    double ans = dot(p[0], z); int id = 0;
    if (dot(p[top], z) > ans) ans = dot(p[
        top], z), id = top;
    int l = 1, r = top - 1;
    while (l < r) {
        int mid = (l + r) >> 1;
        if (dot(p[mid + 1], z) >= dot(p[mid
            ], z)) l = mid + 1;
        else r = mid;
    }
    if (dot(p[l], z) > ans) ans = dot(p[l
        ], z), id = l;
    l = top + 1, r = n - 1;
    while (l < r) {
        int mid = (l + r) >> 1;
        if (dot(p[(mid + 1) % n], z) >= dot(
            p[mid], z)) l = mid + 1;
        else r = mid;
    }
    l %= n;
    if (dot(p[l], z) > ans) ans = dot(p[l
        ], z), id = l;
    return id;
}
pair<PT, int> point_poly_tangent(vector<
    PT> &p, PT Q, int dir, int l, int r
    ) {
    while (r - l > 1) {
        int mid = (l + r) >> 1;
        bool pvs = orientation(Q, p[mid], p[
            mid - 1]) != -dir;
        bool nxt = orientation(Q, p[mid], p[
            mid + 1]) != -dir;
        if (pvs && nxt) return {p[mid], mid
            };
    }
    if (!pvs || !nxt) {
        auto p1 = point_poly_tangent(p, Q,
            dir, mid + 1, r);
        auto p2 = point_poly_tangent(p, Q,
            dir, l, mid - 1);
        return orientation(Q, p1.first, p2
            .first) == dir ? p1 : p2;
    }
    if (!pvs) {
        if (orientation(Q, p[mid], p[l])
            == dir) r = mid - 1;
        else if (orientation(Q, p[l], p[r
            ]) == dir) r = mid - 1;
        else l = mid + 1;
    }
    if (!nxt) {
        if (orientation(Q, p[mid], p[l])
            == dir) l = mid + 1;
        else if (orientation(Q, p[l], p[r
            ]) == dir) r = mid - 1;
        else l = mid + 1;
    }
}
pair<PT, int> ret = {p[l], l};
for (int i = l + 1; i <= r; i++) {
    ret = orientation(Q, ret.first, p[i
        ]) != dir ? make_pair(p[i], i) :
        ret;
}
return ret;
}
// (ccw, cw) tangents from a point that
// is outside this convex polygon
pair<int, int>
    tangents_from_point_to_polygon(
        vector<PT> &p, PT Q) {
    int ccw = point_poly_tangent(p, Q, 1,
        0, (int)p.size() - 1).second;
    int cw = point_poly_tangent(p, Q, -1,
        0, (int)p.size() - 1).second;
    return make_pair(ccw, cw);
}
// Minimum distance from a point to a
// convex polygon (assumes strictly
// outside)
double dist_from_point_to_polygon(vector
    <PT> &p, PT z) {
    double ans = inf;
    int n = p.size();
    if (n <= 3) {
        for (int i = 0; i < n; i++) ans = min
            (ans, dist_from_point_to_seg(p[i],
                p[(i + 1) % n], z));
        return ans;
    }
    auto [r, l] =
        tangents_from_point_to_polygon(p, z
        );
    if (l > r) r += n;
    while (l < r) {
        int mid = (l + r) >> 1;
        double left = dist2(p[mid % n], z),
            right = dist2(p[(mid + 1) % n], z);
        ans = min({ans, left, right});
        if (left < right) r = mid;
        else l = mid + 1;
    }
    ans = sqrt(ans);
    ans = min(ans, dist_from_point_to_seg(
        p[l % n], p[(l + 1) % n], z));
    ans = min(ans, dist_from_point_to_seg(
        p[l % n], p[(l - 1 + n) % n], z));
    return ans;
}
// Minimum distance from convex polygon
// p to line ab (O(log n))
double dist_from_polygon_to_line(vector<
    PT> &p, PT a, PT b, int top) {
    PT orth = (b - a).perp();
    if (orientation(a, b, p[0]) > 0) orth
        = (a - b).perp();
    int id = extreme_vertex(p, orth, top);
    if (dot(p[id] - a, orth) > 0) return
        0.0; // Intersects
    return dist_from_point_to_line(a, b, p
        [id]); // Does not intersect
}
// Minimum distance from a convex

```

```

    polygon to another convex polygon (O
    (n log n))
double dist_from_polygon_to_polygon(
    vector<PT> &p1, vector<PT> &p2) {
    double ans = inf;
    for (int i = 0; i < p1.size(); i++)
        ans = min(ans,
            dist_from_point_to_polygon(p2, p1[i
                ]));
    for (int i = 0; i < p2.size(); i++)
        ans = min(ans,
            dist_from_point_to_polygon(p1, p2[i
                ]));
    return ans;
}
// Maximum distance from a convex
// polygon to another convex polygon (O
// (n))
double
    maximum_dist_from_polygon_to_polygon(
        vector<PT> &u, vector<PT> &v) {
    int n = (int)u.size(), m = (int)v.size
        ();
    double ans = 0;
    if (n < 3 || m < 3) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) ans =
                max(ans, dist2(u[i], v[j]));
        }
        return sqrt(ans);
    }
    if (u[0].x > v[0].x) swap(n, m), swap(
        u, v);
    int i = 0, j = 0, step = n + m + 10;
    while (j + 1 < m && v[j].x < v[j + 1].
        x) j++;
    while (step-- > 0) {
        if (cross(u[(i + 1) % n] - u[i], v[(j
            + 1) % m] - v[j]) >= 0) j = (j + 1) %
            m;
        else i = (i + 1) % n;
        ans = max(ans, dist2(u[i], v[j]));
    }
    return sqrt(ans);
}
// BUGFIX: Using 'double' instead of '
// int' for coordinates to prevent
// precision loss.
double polygon_area(vector<PT> &v) {
    double calc_area = 0;
    int n = v.size();
    for (int i = 0; i < n; i++) {
        PT a = v[i], b = v[(i + 1) % n];
        double x1 = a.x, y1 = a.y; // Fixed
        from int
        double x2 = b.x, y2 = b.y; // Fixed
        from int
        calc_area += (x1 * y2 - x2 * y1);
    }
    return abs(calc_area) / 2.0;
}
// WARNING: It's highly recommended to
// use the standard '
// is_point_in_polygon()'
// above instead of this custom one,
// since 'seg_seg_intersection' returns
// a boolean
// making 'val == 1' or 'val == 2'
// evaluations buggy.
int position_of_a_point_in_a_polygon(
    vector<PT> &v, PT c) {
    double xInf = inf, yInf = c.y + 1;
    PT d(xInf, yInf);
    int n = v.size();
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        PT a = v[i], b = v[(i + 1) % n];
        // This relies on your overridden
        version of seg_seg_intersection
        which returns bool.
        bool intersect =
            seg_seg_intersection(a, b, c, d);
        if (is_point_on_seg(a, b, c)) return
            0; // BOUNDARY
        if (intersect) cnt++;
    }
    if (cnt % 2) return 1; // INSIDE
    else return -1; // OUTSIDE
}

```

```

}
// 0 -> collinear, 1 -> clockwise, 2 ->
// counter-clockwise
int getSign(int x1, int y1, int x2, int
y2, int x3, int y3) {
    int direct = (y2 - y1) * (x3 - x2) - (
y3 - y2) * (x2 - x1);
    if (direct > 0) return 1; //
    clockwise
    if (direct < 0) return 2; //
    counter - Clockwise
    return 0; //
    collinear
}
// 0 -> collinear, 1 -> clockwise, 2 ->
// counter-clockwise
int ori(int x1, int y1, int x2, int y2,
int x3, int y3) {
    int val = (y2 - y1) * (x3 - x2) - (x2
- x1) * (y3 - y2);
    if (val == 0) return 0; //
    collinear
    else if (val < 0) return 2; //
    counter-clockwise
    return 1; //
    clockwise
}

```

11 Miscellaneous

11.1 Interval Container

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). Time: $O(\log N)$

```

set<pii>::iterator addInterval(set<pii>&
is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}),
    before = it;
    while (it != is.end() && it->first <=
R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second
>= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}
void removeInterval(set<pii>& is, int L,
int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
    if (R != r2) is.emplace(R, r2);
}

```

11.2 Efficient Mask Traversal

Description: Traverses the mask with minimal changes in 2^n as opposed to the traditional $n2^n$ approach. If you don't care about time complexity, you can do a greedy brute force with 4^n time complexity.

```

int mask = 0;
for (int curOp = 1; curOp < (1LL << n);
++curOp) {
    int curOp = __builtin_ctz(x);
    mask ^= (1LL << curOp);
}

```

11.3 Generationg Permutations

```

VI a;
//...
sort(all(a));
do {
    // Process permutation...
} while (next_permutation(all(a)));

```

11.4 Iterating over Submasks

```

int n = 17;
for (int mask = 0; mask < (1 << n); ++
mask) {
    for (int sub = mask; sub; sub = (sub -
1) & mask) {
        // Process submask...
    }
}

```

11.5 Random

```

mt19937 rng(chrono::steady_clock::now().
time_since_epoch().count());
int randint(int l, int r) {
    uniform_int_distribution<int> dist(l,
r);
    return dist(rng);
}

```

11.6 Index compression

```

struct compress {
    vector<int> a;
    compress(vector<int>& v) : a(v) {
        sort(all(a));
        a.erase(unique(all(a)), a.end());
    }
    int size() { return a.size(); }
    int operator[](int val) { return
lower_bound(all(a), val) - a.begin
(); }
};

```

11.7 pbds

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type,
less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
ordered_set<int> st;
// ordered multiset
ordered_set<pair<int, int>> multi;
// get index/ number of smaller elements
// than x
st.order_of_key(x);
// access by index
st.find_by_order(idx);

```

11.8 Bitset

```

bitset<N> b; // All 0s; b[i] access; b.
test(i) bounds check
bitset<N> b(string("110")); // b[0]=0, b
[1]=1, b[2]=1
b.set(i, v=1); b.reset(i); b.flip(i); //
Single bit modify
b.set(); b.reset(); b.flip(); // All
bits modify
b.count(); b.any(); b.all(); // popcount
; if 1+ bits set; if all bits set
b >= k; b <= k; b &= b2; b |= b2; b ^=
b2; b = ~b; // O(N/64) bitwise
// GCC Specific (The good stuff):
b._Find_first(); // Index of first '1' (
returns N if none)
b._Find_next(i); // Index of next '1'
after i (returns N if none)

```

11.9 Primes

```

int BASES[] = {31, 37, 41, 43, 47, 53,
59, 61, 67, 71, 137, 139, 149, 151,
157, 257, 263, 269, 271, 277, 311,
313, 317, 331, 347};
int MOD_32[] = {998244353, 999999937,
1000000007, 1000000009, 1000000021,
1000000103, 1000003931,
1000004357, 2147483647};
11 MOD_1E12[] = {10000000000039,
10000000000063, 10000000000091,
10000000000121, 10000000000169,
10000000000213, 10000000000223};
11 MOD_64[] = {100000000000000003,
100000000000000009, 1000000000000000033,
4611686018427387847, 9223372036854775783};

```

```

pair<int, int> ntt_primes = {{(119 <<
23) + 1, 3}, {(5 << 25) + 1, 3},
{(7 << 26) + 1, 3}, {(479 << 21) +
1, 3}, {(483 << 21) + 5, 5}};

```

11.10 Digit Range Sum

Description: Gives the digit sum of the numbers in the range $[l, r]$. I have encountered this problem twice.

```

auto qrsm = [](ll n) -> ll {
    ll ans = 0;
    ll po = 1;
    ll nm = n;
    while (n) {
        ll tm = n / 10;
        ll rem = n % 10;
        ll s1 = rem * (rem - 1) / 2;
        ll past = (nm % po) + 111;
        ans += (tm * 4511) * po + (s1) * (po
) + (rem * past);
        po *= 10;
        n /= 10;
    }
    return ans;
};
auto rsm = [&](ll l, ll r) -> ll {
    return qrsm(r) - qrsm(l - 1);
};

```

11.11 Custom Hash

```

// Usage: unordered_set<pair<int, int>,
// pointhash> st;
struct pointhash {
    size_t operator()(const pair<int, int
>& p) const {
        return hash<int>()(p.first) ^ (hash<
int>()(p.second) << 1);
    }
};
struct custom_hash {
    static uint64_t splitmix64(uint64_t x)
    {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0
        xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0
        x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().
        time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
unordered_map<long long, int,
custom_hash> safe_map;

```

12 Extras

12.1 Primitive Roots

```

// Primitive root modulo p (p must be
// prime) g is primitive if g^(phi/p_i)
// != 1 for all prime factors p_i of
// phi
int PrimitiveRoot(int p){
    vector<int> fact;
    int phi=p-1,n=phi;
    for(int i=2;i*i<=n;i++){
        if(n%i==0){
            fact.push_back(i);
            while(n%i==0)n/=i;
        }
    }
    if(n>1)fact.push_back(n);
    for(int g=2;g<=p;g++){
        bool ok=true;
        for(int f:fact)
            ok&=BigMod(g,phi/f,p)!=1;
        if(ok)return g;
    }
    return -1;
}
// Solve x to power k a (mod n), n
// prime Prints all solutions using
// primitive root + BSGS

```

```

void printDiscreteRoot(int k,int a,int n
){
    if(a==0){cout<<"1\n0\n";return;}
    int g=PrimitiveRoot(n),phi=n-1;
    int sq=(int)sqrt(n)+1;
    vector<pair<int,int>>dec(sq);
    for(int i=1;i<=sq;i++)
        dec[i-1]={BigMod(g,1LL*i*sq%phi*
            k%phi,n),i};
    sort(dec.begin(),dec.end());
    int any=-1;
    for(int i=0;i<=sq;i++){
        int val=1LL*BigMod(g,1LL*i*k%phi
            ,n)*a%n;
        auto it=lower_bound(dec.begin(),
            dec.end(),make_pair(val,0));
        if(it!=dec.end()&&it->first==val
    ){
        any=it->second*sq-i;
        break;
    }
}
if(any==-1){cout<<"0\n";return;}
int delta=phi/__gcd(k,phi);
vector<int>ans;
for(int cur=(any%delta+delta)%delta;
    cur<phi;cur+=delta)
    ans.push_back(powmod(g,cur,n));
sort(ans.begin(),ans.end());
cout<<ans.size()<<"\n";
for(int x:ans)cout<<x<<" ";
}

```

12.2 Closest Pair

```

#define x first
#define y second
long long dist2(pair<int, int> a, pair<
    int, int> b) {
    return 1LL * (a.x - b.x) * (a.x - b.x)
        + 1LL * (a.y - b.y) * (a.y - b.y);
}
pair<int, int> closest_pair(vector<pair<
    int, int>> a) {
    int n = a.size();
    assert(n >= 2);
    vector<pair<pair<int, int>, int>> p(n);
    for (int i = 0; i < n; i++) p[i] = {a[i]
        ], i};
    sort(p.begin(), p.end());
    int l = 0, r = 2;
    long long ans = dist2(p[0].x, p[1].x);
    pair<int, int> ret = {p[0].y, p[1].y};
    while (r < n) {
        while (l < r && 1LL * (p[r].x.x - p[l]
            ].x.x) * (p[r].x.x - p[l].x.x) >=
            ans) l++;
        for (int i = l; i < r; i++) {
            long long nw = dist2(p[i].x, p[r].x);
            if (nw < ans) {
                ans = nw;
                ret = {p[i].y, p[r].y};}r++;}
        return ret;
    }
}

```

12.3 Sweep Line

```

long long sweep_line_x;
struct Segment{long long a,b,x,y;int idx
};
struct Point{long long x,y;int
    segment_idx;};
int sign(long long x){return x==0?0:(x
    <0?-1:1);}
double coordinate(const Segment&s){
    if(s.a==s.x)return s.b;
    return s.b+(double)(s.y-s.b)*(
        sweep_line_x-s.a)/(s.x-s.a);
}
bool operator<(const Segment&u,const
    Segment&v){
    if(u.idx==v.idx)return false;
    return coordinate(u)<coordinate(v);
}
bool operator==(const Segment&u,const
    Segment&v){return u.idx==v.idx;}
long long orientation(Point a,Point b,
    Point c){

```

```

    return (b.x-a.x)*(c.y-a.y)-(c.x-a.x)
        *(b.y-a.y);
}
bool intersect(const Segment&a,const
    Segment&b){
    Point p1{a.a,a.b,0},q1{a.x,a.y,0},p2
        {b.a,b.b,0},q2{b.x,b.y,0};
    long long o1=orientation(p1,q1,p2),
        o2=orientation(p1,q1,q2);
    long long o3=orientation(p2,q2,p1),
        o4=orientation(p2,q2,q1);
    return sign(o1)*sign(o2)<=0&&sign(o3)
        *sign(o4)<=0;
}
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;cin>>n;
    vector<Segment>segments(n);
    vector<Point>events;
    for(int i=0;i<n;i++){
        cin>>segments[i].a>>segments[i].
            b>>segments[i].x>>segments[i].y;
        segments[i].idx=i;
        events.push_back({segments[i].a,
            segments[i].b,i});
        events.push_back({segments[i].x,
            segments[i].y,i});
    }
    sort(events.begin(),events.end(),[](<
        const Point&p,const Point&q){
        if(p.x!=q.x)return p.x<q.x;
        return p.y<q.y;
    });
    set<Segment>active;
    int first=-1,second=-1;
    for(auto&e:events){
        sweep_line_x=e.x;
        int id=e.segment_idx;
        auto it=active.find(segments[id]
    );
        if(it!=active.end()){
            auto nxt=next(it);
            if(it!=active.begin()&&nxt!=
                active.end()){
                auto prv=prev(it);
                if(intersect(segments[
                    prv->idx],segments[nxt->idx])){
                    first=prv->idx;
                    second=nxt->idx;break;
                }
            }
            active.erase(it);
        }else{
            it=active.lower_bound(
                segments[id]);
            if(it!=active.end()&&
                intersect(segments[id],*it)){
                first=id;second=it->idx;
                break;
            }
            if(it!=active.begin()){
                auto prv=prev(it);
                if(intersect(segments[id]
                    ],*prv)){
                    first=id;second=prv
                        ->idx;break;
                }
            }
            active.insert(segments[id]);
        }
    }
    if(first>second)swap(first,second);
    int cnt=0;
    for(int i=0;i<n;i++){
        if(i!=second&&intersect(segments
            [i],segments[second]))cnt++;
        cout<<(cnt>1?second+1:first+1);
    }
}

```

12.4 Pick's Theorem

```

theorem : area = In + (on_boundary / 2)
    - 1;
in = strictly inside polygon
T point(PT a, PT b){
    T x = abs(a.x - b.x);
    T y = abs(a.y - b.y);

```

```

    return __gcd(x, y) - 1;
}
main(){
    ll on_boundary = n, area = 0LL;
    for (i = 0; i < n; i++){
        area += cross(p[i], p[(i + 1) % n]
    );
    on_boundary += point(p[i], p[(i +
        1) % n]);
    }
}

```

13 Formulae

13.0.1 Inclusion-Exclusion Principle

Let S_j be the sum of sizes of all intersections of exactly j properties:

$$S_j = \sum_{1 \leq i_1 < \dots < i_j \leq n} |A_{i_1} \cap \dots \cap A_{i_j}|$$

• Union (At least 1):

$$|A_1 \cup \dots \cup A_n| = \sum_{j=1}^n (-1)^{j-1} S_j$$

• Exactly k properties:

$$E_k = \sum_{j=k}^n (-1)^{j-k} \binom{j}{k} S_j$$

• At least k properties:

$$L_k = \sum_{j=k}^n (-1)^{j-k} \binom{j-1}{k-1} S_j$$

Implementation Note:

If intersections depend only on the number of sets j , precompute $S_j = \binom{n}{j} \times \text{intersect}(j)$ in $O(n)$. Otherwise, use bitmask PIE in $O(2^n)$.

13.0.2 Extgcd and Diophantine

Linear Diophantine equation solves the equation $ax + by = c$ where $\gcd(a, b) \mid c$ Extended gcd calculates $ax + by = 1$ where $\gcd(a, b) = 1$. Suppose, $(x, y) = (x_0, y_0)$. Now we can write diophantine equation as $\frac{a}{g}x + \frac{b}{g}y = \frac{c}{g}$ where $g = \gcd(a, b)$. This equation can be put into extended gcd and we can obtain $(X, Y) = (\frac{c}{g}x_0, \frac{c}{g}y_0)$. Thus the general solution is obtained as:

$$x = X + \frac{b}{g}t \quad y = Y - \frac{a}{g}t \quad t \in \mathbb{Z}$$

Typically these type of problem requires a range of x, y ($x \in [\min x, \max x], y \in [\min y, \max y]$).

13.0.3 Combinatorics

- $\sum_{0 \leq k \leq n} \binom{n-k}{k} = \text{Fib}_{n+1}$
- $\binom{n}{k} = \binom{n}{n-k}$
- $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- $k \binom{n}{k} = n \binom{n-1}{k-1}$
- $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$
- $\sum_{i=0}^n \binom{n}{i} = 2^n$
- $\sum_{i \geq 0} \binom{n}{2i} = 2^{n-1}$
- $\sum_{i \geq 0} \binom{n}{2i+1} = 2^{n-1}$
- $\sum_{i=0}^k (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
- $\sum_{i=0}^k \binom{n+i}{i} = \sum_{i=0}^k \binom{n+i}{n} =$
- $1 \binom{n}{1} + 2 \binom{n}{2} + 3 \binom{n}{3} + \dots + n \binom{n}{n} = n 2^{n-1}$

$$12. 1^2 \binom{n}{1} + 2^2 \binom{n}{2} + 3^2 \binom{n}{3} + \dots + n^2 \binom{n}{n} = (n + n^2)2^{n-2}$$

13. **Vandermonde's Identify:**

$$\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$$

14. **Hockey-Stick Identify:**

$$n, r \in N, n > r, \sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$$

$$15. \sum_{i=0}^k \binom{k}{i}^2 = \binom{2k}{k}$$

$$16. \sum_{k=0}^n \binom{n}{k} \binom{n}{n-k} = \binom{2n}{n}$$

$$17. \sum_{k=q}^n \binom{n}{k} \binom{k}{q} = 2^{n-q} \binom{n}{q}$$

$$18. \sum_{i=0}^n k^i \binom{n}{i} = (k+1)^n$$

$$19. \sum_{i=0}^n \binom{2n}{i} = 2^{2n-1} + \frac{1}{2} \binom{2n}{n}$$

$$20. \sum_{i=1}^n \binom{n}{i} \binom{n-1}{i-1} = \binom{2n-1}{n-1}$$

$$21. \sum_{i=0}^n \binom{2n}{i}^2 = \frac{1}{2} \left(\binom{4n}{2n} + \binom{2n}{n}^2 \right)$$

22. **Highest Power of 2 that divides $2^n C_n$:** Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x . Let it form a sequence. The n -th value in the sequence (starting from $n = 0$) gives the highest power of 2 that divides $2^n C_n$.

23. **Pascal Triangle**

(a) In a row p where p is a prime number, all the terms in that row except the 1s are multiples of p .

(b) Parity: To count odd terms in row n , convert n to binary. Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x .

(c) Every entry in row $2^n - 1, n \geq 0$, is odd.

24. An integer $n \geq 2$ is prime if and only if all the intermediate binomial coefficients

$$\binom{n}{1}, \binom{n}{2}, \dots, \binom{n}{n-1} \text{ are divisible by } n.$$

25. **Kummer's Theorem:** For given integers $n \geq m \geq 0$ and a prime number p , the largest power of p dividing $\binom{n}{m}$ is equal to the number of carries when m is added to $n-m$ in base p . For implementation take inspiration from lucas theorem.

26. Number of different binary sequences of length n such that no two 0's are adjacent = Fib_{n+1}

27. **Combination with repetition:** Let's say we choose k elements from an n -element set, the order doesn't matter and each element can be chosen more than once. In that case, the number of different combinations is: $\binom{n+k-1}{k}$

28. Number of ways to divide n persons in $\frac{n}{k}$ equal groups i.e. each having size k is

$$\frac{n!}{k!^{\frac{n}{k}} \left(\frac{n}{k}\right)!} = \prod_{n \geq k} \binom{n-1}{k-1}$$

29. The number non-negative solution of the equation: $x_1 + x_2 + x_3 + \dots + x_k = n$ is $\binom{n+k-1}{n}$

30. Number of ways to choose n ids from 1 to b such that every id has distance at least $k = \left(\frac{b - (n-1)(k-1)}{n} \right)$

$$31. \sum_{i=1,3,5,\dots}^{i \leq n} \binom{n}{i} a^{n-i} b^i = \frac{1}{2} ((a+b)^n - (a-b)^n)$$

$$32. \sum_{i=0}^n \frac{\binom{k}{i}}{\binom{n}{i}} = \frac{\binom{n+1}{n-k+1}}{\binom{n}{n-k}}$$

33. **Derangement:** a permutation of the elements of a set, such that no element appears in its original position. Let $d(n)$ be the number of derangements of the identity permutation for size n .

$$d(n) = (n-1)(d(n-1) + d(n-2)), \text{ where } d(0) = 1, d(1) = 0.$$

34. **Involutions:** permutations such that $p^2 = \text{identity permutation}$. $a_0 = a_1 = 1$ and $a_n = a_{n-1} + (n-1)a_{n-2}$ for $n > 1$.

35. Let $T(n, k)$ be the number of permutations of size n for which all cycles have length $\leq k$.

$$T(n, k) = \begin{cases} n! & ; n \leq k \\ n \cdot T(n-1, k) & \\ -F(n-1, k) & \\ \cdot T(n-k-1, k) & ; n > k \end{cases}$$

$$\text{Here } F(n, k) = n \cdot (n-1) \cdot \dots \cdot (n-k+1)$$

36. **Lucas Theorem**

(a) If p is prime, then $\left(\frac{p^a}{k} \right) \equiv 0 \pmod{p}$

(b) For non-negative integers m and n and a prime p , the following congruence relation holds: $\left(\frac{m}{n} \right) \equiv \prod_{i=0}^k \left(\frac{m_i}{n_i} \right) \pmod{p}$, where, $m = m_k p^k + m_{k-1} p^{k-1} + \dots + m_1 p + m_0$, and $n = n_k p^k + n_{k-1} p^{k-1} + \dots + n_1 p + n_0$ are the base p expansions of m and n respectively. This uses the convention that $\left(\frac{m}{n} \right) = 0$, when $m < n$.

$$\begin{aligned} 37. \sum_{i=0}^n \binom{n}{i} \cdot i^k &= \sum_{i=0}^n \binom{n}{i} \cdot \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot i^j \\ &= \sum_{i=0}^n \binom{n}{i} \cdot \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot j! \binom{n}{i} \\ &= \sum_{i=0}^n \frac{n!}{(n-i)!} \cdot \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(i-j)!} \\ &= \sum_{i=0}^n \sum_{j=0}^k \frac{n!}{(n-i)!} \cdot \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(i-j)!} \\ &= n! \sum_{i=0}^n \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(n-i)!} \cdot \frac{1}{(i-j)!} \\ &= n! \sum_{i=0}^n \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \binom{n-j}{n-i} \cdot \frac{1}{(n-j)!} \\ &= n! \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(n-j)!} \cdot \sum_{i=0}^n \binom{n-j}{n-i} \\ &= \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot n^j \cdot 2^{n-j} \end{aligned}$$

Here $n^j = P(n, j) = \frac{n!}{(n-j)!}$ and $\left\{ \begin{matrix} k \\ j \end{matrix} \right\}$ is stirling number of the second kind.

So, instead of $O(n)$, now you can calculate the original equation in $O(k^2)$ or even in $O(k \log^2 n)$ using NTT.

$$38. \sum_{i=0}^{n-1} \binom{i}{j} x^i = x^j (1-x)^{-j-1} \cdot \left(1 - x^n \sum_{i=0}^j \binom{n}{i} x^{j-i} (1-x)^i \right)$$

$$39. x_0, x_1, x_2, x_3, \dots, x_n \\ x_0 + x_1, x_1 + x_2, x_2 + x_3, \dots, x_{n-1} + x_n$$

... If we continuously do this n times then the polynomial of the first column of the n -th row will be

$$p(n) = \sum_{k=0}^n \binom{n}{k} \cdot x(k)$$

$$40. \text{ If } P(n) = \sum_{k=0}^n \binom{n}{k} \cdot Q(k), \text{ then,}$$

$$Q(n) = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} \cdot P(k)$$

$$41. \text{ If } P(n) = \sum_{k=0}^n (-1)^k \binom{n}{k} \cdot Q(k), \text{ then,}$$

$$Q(n) = \sum_{k=0}^n (-1)^k \binom{n}{k} \cdot P(k)$$

13.0.4 Catalan Numbers

$$42. C_n = \frac{1}{n+1} \binom{2n}{n}$$

$$43. C_0 = 1, C_1 = 1 \text{ and } C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$$

44. Number of correct bracket sequence consisting of n opening and n closing brackets.

45. The number of ways to completely parenthesize $n+1$ factors.

46. The number of triangulations of a convex polygon with $n+2$ sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).

47. The number of ways to connect the $2n$ points on a circle to form n disjoint i.e. non-intersecting chords.

48. The number of monotonic lattice paths from point $(0,0)$ to point (n,n) in a square lattice of size $n \times n$, which do not pass above the main diagonal (i.e. connecting $(0,0)$ to (n,n)).

49. The number of rooted full binary trees with $n+1$ leaves (vertices are not numbered). A rooted binary tree is full if every vertex has either two children or no children.

50. Number of permutations of $\{1, \dots, n\}$ that avoid the pattern 123 (or any of the other patterns of length 3); that is, the number of permutations with no three-term increasing subsequence. For $n = 3$, these permutations are 132, 213, 231, 312 and 321. For $n = 4$, they are 1432, 2143, 2413, 2431, 3142, 3214, 3241, 3412, 3421, 4132, 4213, 4231, 4312 and 4321.

51. **Balanced Parentheses count with prefix:** The count of balanced parentheses sequences consisting of $n+k$ pairs of parentheses where the first k symbols are open brackets. Let the number be $C_n^{(k)}$, then

$$C_n^{(k)} = \frac{k+1}{n+k+1} \binom{2n+k}{n}$$

13.0.5 Narayana numbers

52. $N(n, k) = \frac{1}{n} \binom{n}{k} \binom{n}{k-1}$
53. The number of expressions containing n pairs of parentheses, which are correctly matched and which contain k distinct nestings. For instance, $N(4, 2) = 6$ as with four pairs of parentheses six sequences can be created which each contain two times the sub-pattern '()'.

13.0.6 Stirling numbers of the first kind

54. The Stirling numbers of the first kind count permutations according to their number of cycles (counting fixed points as cycles of length one).
55. $S(n, k)$ counts the number of permutations of n elements with k disjoint cycles.
56. $S(n, k) = (n-1) \cdot S(n-1, k) + S(n-1, k-1)$, where, $S(0, 0) = 1, S(n, 0) = S(0, n) = 0$
57. $\sum_{k=0}^n S(n, k) = n!$
58. The unsigned Stirling numbers may also be defined algebraically, as the coefficient of the rising factorial:

$$x^{\bar{n}} = x(x+1) \dots (x+n-1) = \sum_{k=0}^n S(n, k) x^k$$

59. Let $[n, k]$ be the stirling number of the first kind, then

$$[n \atop k] = \sum_{0 \leq i_1 < i_2 < \dots < i_k < n} i_1 i_2 \dots i_k.$$

13.0.7 Stirling numbers of the second kind

60. Stirling number of the second kind is the number of ways to partition a set of n objects into k non-empty subsets.
61. $S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$, where $S(0, 0) = 1, S(n, 0) = S(0, n) = 0$
62. $S(n, 2) = 2^{n-1} - 1$
63. $S(n, k) \cdot k! =$ number of ways to color n nodes using colors from 1 to k such that each color is used at least once.
64. An r -associated Stirling number of the second kind is the number of ways to partition a set of n objects into k subsets, with each subset containing at least r elements. It is denoted by $S_r(n, k)$ and obeys the recurrence relation. $S_r(n+1, k) = k S_r(n, k) + \binom{n}{r-1} S_r(n-r+1, k-1)$
65. Denote the n objects to partition by the integers $1, 2, \dots, n$. Define the reduced Stirling numbers of the second kind, denoted $S^d(n, k)$, to be the number of ways to partition the integers $1, 2, \dots, n$ into k nonempty subsets such that all elements in each subset have pairwise distance at least d . That is, for any integers i and j in a given subset, it is required that $|i-j| \geq d$. It has been shown that these numbers satisfy, $S^d(n, k) = S(n-d+1, k-d+1), n \geq k \geq d$

13.0.8 Bell number

66. Counts the number of partitions of a set.
67. $B_{n+1} = \sum_{k=0}^n \binom{n}{k} \cdot B_k$
68. $B_n = \sum_{k=0}^n S(n, k)$, where $S(n, k)$ is stirling number of second kind.

13.0.9 Math

69. $ab \bmod ac = a(b \bmod c)$
70. $\sum_{i=0}^n i \cdot i! = (n+1)! - 1$
71. $a^k - b^k = (a-b) \cdot (a^{k-1}b^0 + a^{k-2}b^1 + \dots + a^0b^{k-1})$
72. $\min(a+b, c) = a + \min(b, c-a)$
73. $|a-b| + |b-c| + |c-a| = 2(\max(a, b, c) - \min(a, b, c))$
74. $a \cdot b \leq c \rightarrow a \leq \left\lfloor \frac{c}{b} \right\rfloor$ is correct
75. $a \cdot b < c \rightarrow a < \left\lfloor \frac{c}{b} \right\rfloor$ is incorrect
76. $a \cdot b \geq c \rightarrow a \geq \left\lfloor \frac{c}{b} \right\rfloor$ is correct
77. $a \cdot b > c \rightarrow a > \left\lfloor \frac{c}{b} \right\rfloor$ is correct
78. For positive integer n , and arbitrary real numbers m, x , $\left\lfloor \frac{\lfloor x/m \rfloor}{n} \right\rfloor = \left\lfloor \frac{x}{mn} \right\rfloor$
79. Lagrange's identity:

$$\begin{aligned} & \left(\sum_{k=1}^n a_k^2 \right) \left(\sum_{k=1}^n b_k^2 \right) - \left(\sum_{k=1}^n a_k b_k \right)^2 \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n (a_i b_j - a_j b_i)^2 \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1, j \neq i}^n (a_i b_j - a_j b_i)^2 \end{aligned}$$

$$80. \sum_{i=1}^n i a^i = \frac{a(na^{n+1} - (n+1)a^n + 1)}{(a-1)^2}$$

81. **Vieta's formulas:** Any general polynomial of degree n

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

(with the coefficients being real or complex numbers and $a_n \neq 0$) is known by the fundamental theorem of algebra to have n (not necessarily distinct) complex roots $r_1, r_2, \dots, r_n$.

$$\begin{cases} r_1 + r_2 + \dots + r_{n-1} + r_n = -\frac{a_{n-1}}{a_n} \\ (r_1 r_2 + r_1 r_3 + \dots + r_1 r_n) \\ + (r_2 r_3 + r_2 r_4 + \dots + r_2 r_n) + \dots \\ + r_{n-1} r_n = \frac{a_{n-2}}{a_n} \\ \vdots \\ r_1 r_2 \dots r_n = (-1)^n \frac{a_0}{a_n}. \end{cases}$$

Vieta's formulas can equivalently be written as

$$\sum_{1 \leq i_1 < i_2 < \dots < i_k \leq n} \left(\prod_{j=1}^k r_{i_j} \right) = (-1)^k \frac{a_{n-k}}{a_n}$$

82. We are given n numbers $a_1, a_2, \dots, a_n$ and our task is to find a value x that minimizes the sum,

$$|a_1 - x| + |a_2 - x| + \dots + |a_n - x|$$

optimal x = median of the array. if n is even $x = [\text{left median}, \text{right median}]$ i.e. every number in this range will work. For minimizing

$$(a_1 - x)^2 + (a_2 - x)^2 + \dots + (a_n - x)^2$$

$$\text{optimal } x = \frac{(a_1 + a_2 + \dots + a_n)}{n}$$

83. Given an array a of n non-negative integers. The task is to find the sum of the product of elements of all the possible subsets. It is equal to the product of $(a_i + 1)$ for all a_i

84. **Pentagonal number theorem:** In mathematics, the pentagonal number theorem states that

$$\prod_{n=1}^{\infty} (1 - x^n) = \sum_{k=-\infty}^{\infty} (-1)^k x^{\frac{k(3k-1)}{2}} = 1 + \sum_{k=1}^{\infty} (-1)^k \left(x^{\frac{k(3k+1)}{2}} + x^{\frac{k(3k-1)}{2}} \right).$$

In other words,

$$\begin{aligned} & (1-x)(1-x^2)(1-x^3) \dots \\ &= 1 - x - x^2 + x^5 + x^7 - x^{12} \\ & \quad - x^{15} + x^{22} + x^{26} - \dots \end{aligned}$$

The exponents $1, 2, 5, 7, 12, \dots$ on the right hand side are given by the formula $g_k = \frac{k(3k-1)}{2}$ for $k = 1, -1, 2, -2, 3, \dots$ and are called (generalized) pentagonal numbers. It is useful to find the partition number in $O(n\sqrt{n})$

13.0.10 Fibonacci Number

85. $F_0 = 0, F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$

$$86. F_n = \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n-k-1}{k}$$

$$87. F_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n$$

$$88. \sum_{i=1}^n F_i = F_{n+2} - 1$$

$$89. \sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$$

$$90. \sum_{i=1}^n F_{2i} = F_{2n+1} - 1$$

$$91. \sum_{i=1}^n F_i^2 = F_n F_{n+1}$$

$$92. F_m F_{n+1} - F_{m-1} F_n = (-1)^n F_{m-n}$$

$$F_{2n} = F_{n+1}^2 - F_{n-1}^2 = F_n (F_{n+1} + F_{n-1})$$

$$93. F_m F_n + F_{m-1} F_{n-1} = F_{m+n-1}$$

$$F_m F_{n+1} + F_{m-1} F_n = F_{m+n}$$

94. A number is Fibonacci if and only if one or both of $(5 \cdot n^2 + 4)$ or $(5 \cdot n^2 - 4)$ is a perfect square

95. Every third number of the sequence is even and more generally, every k^{th} number of the sequence is a multiple of F_k

$$96. \gcd(F_m, F_n) = F_{\gcd(m, n)}$$

97. Any three consecutive Fibonacci numbers are pairwise coprime, which means that, for every n , $\gcd(F_n, F_{n+1}) = \gcd(F_n, F_{n+2}), \gcd(F_{n+1}, F_{n+2}) = 1$

98. If the members of the Fibonacci sequence are taken $\bmod n$, the resulting sequence is periodic with period at most $6n$.

13.0.11 Pythagorean Triples

99. A Pythagorean triple consists of three positive integers a, b , and c , such that $a^2 + b^2 = c^2$. Such a triple is commonly written (a, b, c)

100. Euclid's formula is a fundamental formula for generating Pythagorean triples given an arbitrary pair of integers m and n with $m > n > 0$. The formula states that the integers

$$a = m^2 - n^2, b = 2mn, c = m^2 + n^2$$

form a Pythagorean triple. The triple generated by Euclid's formula is primitive if and only if m and n are coprime and not both odd. When both m and n are odd, then a, b , and c will be even, and the triple will not be primitive; however, dividing a, b , and c by 2 will yield a primitive triple when m and n are coprime and both odd.

101. The following will generate all Pythagorean triples uniquely:

$$\begin{aligned} a &= k(m^2 - n^2), \\ b &= k(2mn), c = k(m^2 + n^2). \end{aligned}$$

where m, n , and k are positive integers with $m > n$, and with m and n coprime and not both odd.

102. **Theorem:** The number of Pythagorean triples a, b, n with $\max(a, b, n) = n$ is given by

$$\frac{1}{2} \left(\prod_{p^\alpha || n} (2\alpha + 1) - 1 \right)$$

where the product is over all prime divisors p of the form $4k + 1$. The notation $p^\alpha || n$ stands for the highest exponent α for which p^α divides n . **Example:** For $n = 2 \cdot 3^2 \cdot 5^3 \cdot 7^4 \cdot 11^5 \cdot 13^6$, the number of Pythagorean triples with hypotenuse n is $\frac{1}{2} (7 \cdot 13 - 1) = 45$. To obtain a formula for the number of Pythagorean triples with hypotenuse less than a specific positive integer N , we may add the numbers corresponding to each $n < N$ given by the Theorem. There is no simple way to compute this as a function of N .

13.0.12 Sum of Squares Function

103. The function is defined as

$$r_k(n) = \left| \left\{ (a_1, a_2, \dots, a_k) \in \mathbf{Z}^k : n = a_1^2 + a_2^2 + \dots + a_k^2 \right\} \right|$$

104. The number of ways to write a natural number as sum of two squares is given by $r_2(n)$. It is given explicitly by $r_2(n) = 4(d_1(n) - d_3(n))$ where $d_1(n)$ is the number of divisors of n which are congruent with 1 modulo 4 and $d_3(n)$ is the number of divisors of n which are congruent with 3 modulo 4. The prime factorization $n = 2^g p_1^{f_1} p_2^{f_2} \dots d_1^{h_1} d_2^{h_2} \dots$, where p_i are the prime factors of the form $p_i \equiv 1 \pmod{4}$, and q_i are the prime factors of the form $q_i \equiv 3 \pmod{4}$ gives another formula $r_2(n) = 4(f_1 + 1)(f_2 + 1) \dots$, if all exponents $h_1, h_2, \dots$ are even. If one or more h_i are odd, then $r_2(n) = 0$.

105. The number of ways to represent n as the sum of four squares is eight times the sum of all its divisors which are not divisible by 4, i.e. $r_4(n) = 8 \sum_{d|n; 4 \nmid d} d$

$$r_8(n) = 16 \sum_{d|n} (-1)^{n+d} d^3$$

13.0.13 Number Theory

106. for $i > j$, $\gcd(i, j) = \gcd(i - j, j) \leq (i - j)$

$$107. \sum_{x=1}^n [d|x^k] = \left\lfloor \frac{n}{\prod_{i=0}^k p_i \left\lceil \frac{e_i}{k} \right\rceil} \right\rfloor, \text{ where } d = \prod_{i=0}^k p_i^{e_i}.$$

Here, $[a|b]$ means if a divides b then it is 1, otherwise it is 0.

108. The number of lattice points on segment (x_1, y_1) to (x_2, y_2) is $\gcd(\text{abs}(x_1 - x_2), \text{abs}(y_1 - y_2)) + 1$
109. $(n - 1)! \pmod n = n - 1$ if n is prime, 2 if $n = 4$, 0 otherwise.
110. A number has odd number of divisors if it is perfect square
111. The sum of all divisors of a natural number n is odd if and only if $n = 2^r \cdot k^2$ where r is non-negative and k is positive integer.
112. Let a and b be coprime positive integers, and find integers a' and b' such that $aa' \equiv 1 \pmod b$ and $bb' \equiv 1 \pmod a$. Then the number of representations of a

positive integers (n) as a non negative linear combination of a and b is

$$\frac{n}{ab} - \left\{ \frac{b/n}{a} \right\} - \left\{ \frac{a/n}{b} \right\} + 1$$

Here, x denotes the fractional part of x .

$$\begin{aligned} & \sum_{i=1}^a \sum_{j=1}^b \sum_{k=1}^c d(i \cdot j \cdot k) \\ &= \sum_{\gcd(i,j)=\gcd(j,k)=\gcd(k,i)=1} \left\lfloor \frac{a}{i} \right\rfloor \left\lfloor \frac{b}{j} \right\rfloor \left\lfloor \frac{c}{k} \right\rfloor \end{aligned}$$

Here, $d(x)$ = number of divisors of x .

113. **Gauss's generalization of Wilson's theorem:**, Gauss proved that,

$$\begin{aligned} & \prod_{\substack{k=1 \\ \gcd(k,m)=1}}^m k \\ & \equiv \begin{cases} -1 \pmod m & \text{if } m = 4, p^\alpha, 2p^\alpha \\ 1 \pmod m & \text{otherwise} \end{cases} \end{aligned}$$

where p represents an odd prime and α a positive integer. The values of m for which the product is -1 are precisely the ones where there is a primitive root modulo m .

13.0.14 Divisor Function

$$114. \sigma_x(n) = \sum_{d|n} d^x$$

115. It is multiplicative i.e if $\gcd(a, b) = 1 \rightarrow \sigma_x(ab) = \sigma_x(a)\sigma_x(b)$.

- 116.

$$\sigma_x(n) = \prod_{i=1}^{\tau} \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$$

117. Divisor Summatory Function

- (a) Let $\sigma_0(k)$ be the number of divisors of k .

$$(b) D(x) = \sum_{n \leq x} \sigma_0(n)$$

$$(c) D(x) = \sum_{k=1}^x \left\lfloor \frac{x}{k} \right\rfloor = 2 \sum_{k=1}^u \left\lfloor \frac{x}{k} \right\rfloor - u^2, \text{ where } u = \sqrt{x}$$

- (d) $D(n)$ = Number of increasing arithmetic progressions where $n + 1$ is the second or later term. (i.e. The last term, starting term can be any positive integer $\leq n$. For example, $D(3) = 5$ and there are 5 such arithmetic progressions: $(1, 2, 3, 4); (2, 3, 4); (1, 4); (2, 4); (3, 4)$.

118. Let $\sigma_1(k)$ be the sum of divisors of k .

$$\text{Then, } \sum_{k=1}^n \sigma_1(k) = \sum_{k=1}^n k \left\lfloor \frac{n}{k} \right\rfloor$$

119. $\prod_{d|n} d = n^{\frac{\sigma_0}{2}}$ if n is not a perfect square, and $= \sqrt{n} \cdot n^{\frac{\sigma_0-1}{2}}$ if n is a perfect square.

13.0.15 Euler's Totient function

120. The function is multiplicative. This means that if $\gcd(m, n) = 1$, $\phi(m \cdot n) = \phi(m) \cdot \phi(n)$.

$$121. \phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

122. If p is prime and $(k \geq 1)$, then: $\phi(p^k) = p^{k-1}(p - 1) = p^k \left(1 - \frac{1}{p}\right)$

123. $J_k(n)$, the Jordan totient function, is the number of k -tuples of positive integers all less than or equal to n that form a coprime $(k + 1)$ -tuple together with n . It is a generalization of Euler's totient, $\phi(n) = J_1(n)$. $J_k(n) = n^k \prod_{p|n} \left(1 - \frac{1}{p^k}\right)$

$$124. \sum_{d|n} J_k(d) = n^k$$

$$125. \sum_{d|n} \phi(d) = n$$

$$126. \phi(n) = \sum_{d|n} \mu(d) \cdot \frac{n}{d} = n \sum_{d|n} \frac{\mu(d)}{d}$$

$$127. \phi(n) = \sum_{d|n} d \cdot \mu\left(\frac{n}{d}\right)$$

$$128. a|b \rightarrow \varphi(a)|\varphi(b)$$

$$129. n|\varphi(a^n - 1) \text{ for } a, n > 1$$

$$130. \varphi(mn) = \varphi(m)\varphi(n) \cdot \frac{d}{\varphi(d)} \text{ where } d = \gcd(m, n) \text{ Note the special cases}$$

$$\varphi(2m) = \begin{cases} 2\varphi(m) & \text{if } m \text{ is even} \\ \varphi(m) & \text{if } m \text{ is odd} \end{cases}$$

$$\varphi(n^m) = n^{m-1}\varphi(n)$$

131. $\varphi(\text{lcm}(m, n)) \cdot \varphi(\gcd(m, n)) = \varphi(m) \cdot \varphi(n)$
Compare this to the formula $\text{lcm}(m, n) \cdot \gcd(m, n) = m \cdot n$

132. $\varphi(n)$ is even for $n \geq 3$. Moreover, if n has r distinct odd prime factors, $2^r | \varphi(n)$

$$133. \sum_{d|n} \frac{\mu^2(d)}{\varphi(d)} = \frac{n}{\varphi(n)}$$

$$134. \sum_{1 \leq k \leq n, \gcd(k, n)=1} k = \frac{1}{2} n \varphi(n) \text{ for } n > 1$$

$$135. \frac{\varphi(n)}{n} = \frac{\varphi(\text{rad}(n))}{\text{rad}(n)} \text{ where } \text{rad}(n) = \prod_{p|n, p \text{ prime}} p$$

$$136. \phi(m) \geq \log_2 m$$

$$137. \phi(\phi(m)) \leq \frac{m}{2}$$

138. When $x \geq \log_2 m$, then

$$n^x \pmod m = n^{\phi(m)+x \pmod{\phi(m)}} \pmod m$$

139. $\sum_{1 \leq k \leq n, \gcd(k, n)=1} \gcd(k - 1, n) = \varphi(n)d(n)$ where $d(n)$ is number of divisors. Same equation for $\gcd(a \cdot k - 1, n)$ where a and n are coprime.

140. For every n there is at least one other integer $m \neq n$ such that $\varphi(m) = \varphi(n)$.

$$141. \sum_{i=1}^n \varphi(i) \cdot \left\lfloor \frac{n}{i} \right\rfloor = \frac{n * (n + 1)}{2}$$

$$142. \sum_{i=1, i \% 2 \neq 0}^n \varphi(i) \cdot \left\lfloor \frac{n}{i} \right\rfloor = \sum_{k \geq 1} \left\lfloor \frac{n}{2k} \right\rfloor^2. \text{ Note that } \lfloor \cdot \rfloor \text{ is used here to denote round operator not floor or ceil}$$

- 143.

$$\sum_{i=1}^n \sum_{j=1}^n ij [\gcd(i, j) = 1] = \sum_{i=1}^n \varphi(i) i^2$$

144. Average of coprimes of n which are less than n is $\frac{n}{2}$.

13.0.16 Mobius Function and Inversion

145. It is a multiplicative function.

146.

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & ; n = 1 \\ 0 & ; n > 0 \end{cases}$$

147. $\sum_{n=1}^N \mu^2(n) = \sum_{n=1}^{\sqrt{N}} \mu(k) \cdot \left\lfloor \frac{N}{k^2} \right\rfloor$ This is also the number of square-free numbers $\leq n$

148. **Mobius inversion theorem:** The classic version states that if g and f are arithmetic functions satisfying $g(n) = \sum_{d|n} f(d)$ for every integer $n \geq 1$ then $g(n) = \sum_{d|n} \mu(d)g\left(\frac{n}{d}\right)$ for every integer $n \geq 1$

149. If $F(n) = \prod_{d|n} f(d)$, then $F(n) = \prod_{d|n} F\left(\frac{n}{d}\right)^{\mu(d)}$

150. $\sum_{d|n} \mu(d)\phi(d) = \prod_{j=1}^K (2 - P_j)$ where p_j is the primes factorization of d

151. If $F(n)$ is multiplicative, $F \not\equiv 0$, then $\sum_{d|n} \mu(d)f(d) = \prod_{i=1} (1 - f(P_i))$ where p_i are primes of n .

13.0.17 GCD and LCM

152. $\gcd(a, 0) = a$

153. $\gcd(a, b) = \gcd(b, a \bmod b)$

154. Every common divisor of a and b is a divisor of $\gcd(a, b)$.

155. if m is any integer, then $\gcd(a + m \cdot b, b) = \gcd(a, b)$

156. The gcd is a multiplicative function in the following sense: if a_1 and a_2 are relatively prime, then $\gcd(a_1 \cdot a_2, b) = \gcd(a_1, b) \cdot \gcd(a_2, b)$.

157. $\gcd(a, b) \cdot \text{lcm}(a, b) = |a \cdot b|$

158. $\gcd(a, \text{lcm}(b, c)) = \text{lcm}(\gcd(a, b), \gcd(a, c))$.

159. $\text{lcm}(a, \gcd(b, c)) = \gcd(\text{lcm}(a, b), \text{lcm}(a, c))$.

160. For non-negative integers a and b , where a and b are not both zero, $\gcd(n^a - 1, n^b - 1) = n^{\gcd(a, b)} - 1$

161. $\gcd(a, b) = \sum_{k|a \text{ and } k|b} \phi(k)$

162. $\sum_{i=1}^n [\gcd(i, n) = k] = \phi\left(\frac{n}{k}\right)$

163. $\sum_{k=1}^n \gcd(k, n) = \sum_{d|n} d \cdot \phi\left(\frac{n}{d}\right)$

164. $\sum_{k=1}^n x^{\gcd(k, n)} = \sum_{d|n} x^d \cdot \phi\left(\frac{n}{d}\right)$

165. $\sum_{k=1}^n \frac{1}{\gcd(k, n)} = \sum_{d|n} \frac{1}{d} \cdot \phi\left(\frac{n}{d}\right) = \frac{1}{n} \sum_{d|n} d \cdot \phi(d)$

166. $\sum_{k=1}^n \frac{k}{\gcd(k, n)} = \frac{n}{2} \cdot \sum_{d|n} \frac{1}{d} \cdot \phi\left(\frac{n}{d}\right) = \frac{n}{2} \cdot \frac{1}{n} \cdot \sum_{d|n} d \cdot \phi(d)$

167. $\sum_{k=1}^n \frac{n}{\gcd(k, n)} = 2 * \sum_{k=1}^n \frac{k}{\gcd(k, n)} - 1$, for $n > 1$

168. $\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$

169. $\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{d=1}^n \phi(d) \left\lfloor \frac{n}{d} \right\rfloor^2$

170. $\sum_{i=1}^n \sum_{j=1}^n i \cdot j [\gcd(i, j) = 1] = \sum_{i=1}^n \phi(i) i^2$

171. $F(n) = \sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = \sum_{l=1}^n \left(\frac{(1 + \lfloor \frac{n}{l} \rfloor) (\lfloor \frac{n}{l} \rfloor)}{2} \right)^2 \sum_{d|l} \mu(d) l d$

172. $\gcd(\text{lcm}(a, b), \text{lcm}(b, c), \text{lcm}(a, c)) = \text{lcm}(\gcd(a, b), \gcd(b, c), \gcd(a, c))$

173. $\gcd(A_L, A_{L+1}, \dots, A_R) = \gcd(A_L, A_{L+1} - A_L, \dots, A_R - A_{R-1})$.

174. Given n , If $SUM = LCM(1, n) + LCM(2, n) + \dots + LCM(n, n)$ then $SUM = \frac{n}{2} (\sum_{d|n} (\phi(d) \times d) + 1)$

13.0.18 Legendre Symbol

175. $\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p}$ and $\left(\frac{a}{p}\right) \in \{-1, 0, 1\}$.

176. The Legendre symbol is periodic in its first (or top) argument: if $a \equiv b \pmod{p}$, then $\left(\frac{a}{p}\right) = \left(\frac{b}{p}\right)$.

177. The Legendre symbol is a completely multiplicative function of its top argument: $\left(\frac{ab}{p}\right) = \left(\frac{a}{p}\right) \left(\frac{b}{p}\right)$

178. The Fibonacci numbers $1, 1, 2, 3, 5, 8, 13, \dots$ are defined by the recurrence $F_1 = F_2 = 1, F_{n+1} = F_n + F_{n-1}$. If p is a prime number then $F_{p-(\frac{p}{5})} \equiv 0 \pmod{p}$, $F_p \equiv \left(\frac{p}{5}\right) \pmod{p}$.

179. Continuing from previous point, $\left(\frac{p}{5}\right) =$ infinite concatenation of the sequence $(1, -1, -1, 1, 0)$ from $p \geq 1$.

180. If $n = k^2$ is perfect square then $\left(\frac{n}{p}\right) = 1$ for every odd prime except $\left(\frac{n}{k}\right) = 0$ if k is an odd prime.

13.0.19 Miscellaneous

181. $a + b = a \oplus b + 2(a \& b)$.

182. $a + b = a | b + a \& b$

183. $a \oplus b = a | b - a \& b$

184. k_{th} bit is set in x iff $x \bmod 2^{k-1} \geq 2^k$. It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.

185. k_{th} bit is set in x iff $x \bmod 2^{k-1} - x \bmod 2^k \neq 0$ ($= 2^k$ to be exact). It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.

186. $n \bmod 2^i = n \& (2^i - 1)$

187. $1 \oplus 2 \oplus 3 \oplus \dots \oplus (4k - 1) = 0$ for any $k \geq 0$

188. **Erdos Gallai Theorem:** The degree sequence of an undirected graph is the non-increasing sequence of its vertex degrees. A sequence of non-negative integers $d_1 \geq d_2 \geq \dots \geq d_n$ can be represented as the degree sequence of finite

simple graph on n vertices if and only if $d_1 + d_2 + \dots + d_n$ is even and

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$$

holds for every k in $1 \leq k \leq n$.

$$189. f_n = \sum_{i=0}^n \binom{n}{i} g_i \Rightarrow g_n = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} f_i$$

13.0.20 Traingle

• **Sine Rule:** $\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$

• **Cosine Rule:** $c^2 = a^2 + b^2 - 2ab \cos C$

• **Area:** $\Delta = \sqrt{s(s-a)(s-b)(s-c)} = \frac{1}{2} bc \sin A$

• **Circumradius (R):** $R = \frac{abc}{4\Delta} = \frac{a}{2 \sin A}$

• **Inradius (r):** $r = \frac{\Delta}{s}$

• **Ex-radii (r_a, r_b, r_c):** $r_a = \frac{\Delta}{s-a}$

• **Centroid (G):** $\left(\frac{\sum x}{3}, \frac{\sum y}{3}\right)$

• **Incenter (I):** $\left(\frac{ax_1+bx_2+cx_3}{a+b+c}, \frac{ay_1+by_2+cy_3}{a+b+c}\right)$

• **Excenter (I_A):** Replace a with $-a$ in Incenter formula.

• **Euler Line:** O, G, H are collinear; $HG = 2GO$.

• **Median (m_a):** $4m_a^2 = 2b^2 + 2c^2 - a^2$ (Apollonius)

• **Angle Bisector (l_a):** $l_a = \frac{2bc}{b+c} \cos(A/2)$

• **Stewart's Theorem:** Cevian (a line from A to a) length d dividing a into m, n :

$$b^2 m + c^2 n = a(d^2 + mn)$$

• **Centroid (G):** Intersection of *medians* (vertex to midpoint).

• **Incenter (I):** Intersection of *angle bisectors*.

• **Circumcenter (O):** Intersection of *perpendicular bisectors* of the sides.

• **Orthocenter (H):** Intersection of *altitudes* (vertex perpendicular to opposite side).

13.0.21 Circle

• **General Eq:** $x^2 + y^2 + 2gx + 2fy + c = 0$. Center $(-g, -f)$, $r = \sqrt{g^2 + f^2 - c}$.

• **Tangent at $P(x_1, y_1)$:** $xx_1 + yy_1 + g(x + x_1) + f(y + y_1) + c = 0$

• **Slope of tangent from $P(x, y)$:** $y + f = m(x + g) \pm r\sqrt{1 + m^2}$

• **Tangency Condition ($y = mx + c$):** $c^2 = r^2(1 + m^2)$

• **Chord Length:** $L = 2\sqrt{r^2 - d^2}$ (d = dist to center)

• **Power of Point P :** $d^2 - r^2$ (d = dist P to center) (Useful for determining if a point is inside or outside)

• **Secant Theorem:** $PA \cdot PB = PC \cdot PD$ (AB and CD are 2 lines drawn from point P over the circle)

• **Sector Area:** $\frac{1}{2} r^2 \theta$ (θ in radians)

• **Segment Area (Cut by chord):** $\frac{1}{2} r^2 (\theta - \sin \theta)$

13.0.22 Points & 2D Line

- **Dist. point** (x_1, y_1) **to line** $ax + by + c = 0$:

$$d = \frac{|ax_1 + by_1 + c|}{\sqrt{a^2 + b^2}}$$

- **Dist. between parallel lines:** $d = \frac{|c_1 - c_2|}{\sqrt{a^2 + b^2}}$

- **Shoelace Area:** $\frac{1}{2} |\sum (x_i y_{i+1} - x_{i+1} y_i)|$

- **foot of Perpendicular** (h, k) :

$$\frac{h - x_1}{a} = \frac{k - y_1}{b} = -\frac{ax_1 + by_1 + c}{a^2 + b^2}$$

- **Reflection** (h, k) : Replace $-(...)$ with $-2(...)$ above.
- **Rotation of Axes (CCW θ):** $x = X \cos \theta - Y \sin \theta$, $y = X \sin \theta + Y \cos \theta$

13.0.23 Vector

- **Direction Cosines:** $l = \cos \alpha$, $m = \cos \beta$, $n = \cos \gamma$. $l^2 + m^2 + n^2 = 1$.

- **Dot Product:** $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$

- **Projection of a on b:** $(\mathbf{a} \cdot \hat{\mathbf{b}}) \hat{\mathbf{b}}$

- **Cross Product:** Area $\triangle = \frac{1}{2} |\mathbf{a} \times \mathbf{b}|$.

- **Scalar Triple (Vol):** $[\mathbf{abc}] = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$. Tet Vol = $\frac{1}{6} |[\dots]|$.

- **Vector Triple:** $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \cdot \mathbf{c}) \mathbf{b} - (\mathbf{a} \cdot \mathbf{b}) \mathbf{c}$

13.0.24 3D Line

Line 1: $\mathbf{r} = \mathbf{a}_1 + \lambda \mathbf{b}_1$. Line 2: $\mathbf{r} = \mathbf{a}_2 + \mu \mathbf{b}_2$.

- **Angle Between Lines:** $\cos \theta = \frac{|\mathbf{b}_1 \cdot \mathbf{b}_2|}{|\mathbf{b}_1| |\mathbf{b}_2|}$

- **Condition for Intersection:** Scalar Triple Product $[\mathbf{a}_2 - \mathbf{a}_1, \mathbf{b}_1, \mathbf{b}_2] = 0$.

- **Intersection Point:** Solve $\mathbf{a}_1 + \lambda \mathbf{b}_1 = \mathbf{a}_2 + \mu \mathbf{b}_2$ for λ, μ .

- **Shortest Dist (Skew):** $D = \frac{|(\mathbf{a}_2 - \mathbf{a}_1) \cdot (\mathbf{b}_1 \times \mathbf{b}_2)|}{|\mathbf{b}_1 \times \mathbf{b}_2|}$

- **Dist Between Parallel Lines:** $D = \frac{|(\mathbf{a}_2 - \mathbf{a}_1) \times \mathbf{b}|}{|\mathbf{b}|}$

- **Dist Point P to Line:** $D = \frac{|(\mathbf{p} - \mathbf{a}) \times \mathbf{b}|}{|\mathbf{b}|}$

- **Foot of Perp F from P:** $\mathbf{f} = \mathbf{a} + \left(\frac{(\mathbf{p} - \mathbf{a}) \cdot \mathbf{b}}{|\mathbf{b}|^2} \right) \mathbf{b}$

- **Image P' in Line:** $\mathbf{p}' = 2\mathbf{f} - \mathbf{p}$.

13.0.25 Plane

Through $\mathbf{a}$, normal $\mathbf{n}$. Eq: $(\mathbf{r} - \mathbf{a}) \cdot \mathbf{n} = 0$.

- **Cartesian:** $Ax + By + Cz + D = 0$. Normal (A, B, C) .

- **Dist Point P to Plane:** $D = \frac{|Ax_1 + By_1 + Cz_1 + D|}{\sqrt{A^2 + B^2 + C^2}}$

- **Angle Between Planes:** $\cos \theta = \frac{|\mathbf{n}_1 \cdot \mathbf{n}_2|}{|\mathbf{n}_1| |\mathbf{n}_2|}$

- **Angle Between Line & Plane:** $\sin \phi = \frac{|\mathbf{b} \cdot \mathbf{n}|}{|\mathbf{b}| |\mathbf{n}|}$

- **Intersection (Line of two planes):** Direction $\mathbf{v} = \mathbf{n}_1 \times \mathbf{n}_2$.

- **Family of Planes:** $P_1 + \lambda P_2 = 0$ (Planes through intersection line).

- **Foot of Perp from P:** $\frac{x - x_1}{A} = \frac{y - y_1}{B} = \frac{z - z_1}{C} = \frac{-(Ax_1 + By_1 + Cz_1 + D)}{A^2 + B^2 + C^2}$

- **Image of Point in Plane:** Use -2 instead of -1 above.

- **Plane Bisectors:** $\frac{Ax_1 + By_1 + Cz_1 + D_1}{\sqrt{A_1^2 + B_1^2 + C_1^2}} = \pm \frac{Ax_2 + By_2 + Cz_2 + D_2}{\sqrt{A_2^2 + B_2^2 + C_2^2}}$

13.0.26 Sphere

- **Standard:** $(x - u)^2 + (y - v)^2 + (z - w)^2 = r^2$.

- **Diameter Form:** $(x - x_1)(x - x_2) + (y - y_1)(y - y_2) + (z - z_1)(z - z_2) = 0$.

- **Tangent Plane at P:** $(\mathbf{r} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) = r^2$.

- **Circle of Intersection:** Radius $R = \sqrt{r^2 - d^2}$.

- **Spherical Cosine Rule:** $\cos a = \cos b \cos c + \sin b \sin c \cos A$.

13.0.27 Solid

- **Cone:** $V = \frac{1}{3} \pi r^2 h$, $SA = \pi r(r + l)$

- **Sphere:** $V = \frac{4}{3} \pi r^3$, $SA = 4\pi r^2$

- **Spherical Cap:** $SA = 2\pi r h$. $V = \frac{1}{3} \pi h^2 (3r - h)$.

- **Frustum (Cone):** $V = \frac{1}{3} \pi h (R^2 + r^2 + Rr)$. $LSA = \pi (R + r) l$.

- **Pyramid:** $V = \frac{1}{3} (\text{Base Area}) \times h$. $LSA = \frac{1}{2} (\text{Perimeter}) \times l$.

- **Regular Tetrahedron:** Side a . $H = a\sqrt{2/3}$. $V = \frac{a^3}{6\sqrt{2}}$. Face Angle $\cos \theta = 1/3$.

13.0.28 Trigonometry

- $\sin(A \pm B) = \sin A \cos B \pm \cos A \sin B$

- $\cos(A \pm B) = \cos A \cos B \mp \sin A \sin B$

- $\cos 2A = \cos^2 A - \sin^2 A = 2 \cos^2 A - 1$

- $\tan^{-1} x \pm \tan^{-1} y = \tan^{-1} \left(\frac{x \pm y}{1 \mp xy} \right)$ (Note: For $+$, if $xy > 1$, add π to the result.)

- $2 \tan^{-1} x = \tan^{-1} \left(\frac{2x}{1 - x^2} \right)$
 $\sin^{-1} \left(\frac{2x}{1 + x^2} \right) = \cos^{-1} \left(\frac{1 - x^2}{1 + x^2} \right)$

- $\tan^{-1} x + \cot^{-1} x = \frac{\pi}{2}$